LLM Prompts for Partial Replication Project

Prompt 1

Prompt: I have an econometrics project to replicate a project, I am doing a descriptive analysis of the dataset, can you check my code is ok please, I have also attached the dataset.

Answer: The code mostly works and is appropriate for the descriptive-analysis section. It loads and sorts the panel data, checks duplicate country-year observations, reports missing values, checks whether each country has at least three consecutive observations for WR and LS, creates graphs for the number of countries observed per year, checks holes inside country-specific spans, produces descriptive statistics and correlations, and creates within, between, first-difference, and two-way fixed effects transformations. The main suggested improvements were to add a table showing missing dependent variables by country, add a clear sample-selection comment table, make the scatterplot regression line safer by checking variation in the x-variable, and note that manual two-way demeaning is fine for descriptive work but panel estimators should be used for regression analysis.

Prompt 2

Prompt: Here are some outputs I got and how I answered the questions. Can you check they are accurate and actually answer it? I just need to do the comment for Table 5.

Answer: The section on sample selection within the unbalanced panel is accurate. The interpretation that WR and LS have 14 countries from 1970 to 1979 and 15 countries from 1980 to 2019 because Japan enters later is correct. The unbalancedness is mild and occurs at the beginning of the sample, with no attrition at the end and no holes inside country-specific time series. Table 4 is also correct: 12 variables vary by country and time, there are no purely time-invariant variables, and PCOM is the only individual-invariant/common time-series variable. For Table 5, the only issue is ordering: if dependent variables must appear first, WR and LS can stay at the top; otherwise the remaining variables should be sorted by within-variance share. The comment should emphasize that WR and LS have substantial within-country variation, i has a very high within share, and variables such as GDP, SHORTUN, and LF have low within shares, meaning that much of their variation reflects persistent differences between countries.

Prompt 3

Prompt: I did this for Table 5 and the question said dependent variables first which is why WR and LS are where they are. Is this good?

Answer: The paragraph is good and answers the Table 5 question. The main corrections are to use 'within-country' with a hyphen, avoid saying that high-within-share variables are necessarily 'common time-series variables', and avoid repeating 'short-term interest rates' twice. A cleaner version states that WR has a within-variance share of 79.01%, LS has 61.56%, and i has 92.70%. It should also say that P, W, and SH have within shares above 90%, while GDP, SHORTUN, and LF have low within shares. The final warning is that high within shares may also reflect a few high-leverage observations after transformation, especially for WR and interest-rate variables.

Prompt 4

Prompt: Changed code to make it more structured. How is it now and help answer Question 6.

Answer: The new code is much more structured and creates the necessary outputs for Question 6: between and within transformed variables, distribution graphs, descriptive statistics, and bivariate correlations. Question 6 can be answered by explaining that the between transformation uses country averages while the one-way within transformation subtracts the country mean. The answer should compare distributions for WR, LS, and i, noting non-normality, skewness, kurtosis, and possible high-leverage observations. The bivariate results show that the between and within correlations with i differ: WR has a strongly positive between correlation with i but a weak negative within correlation; LS has a negative between correlation but a positive within correlation. This shows that cross-country relationships and within-country time relationships tell different stories.

Prompt 5

Prompt: These are my outputs. Help with description for them according to Question 6.

Answer: The outputs are good for Question 6. The write-up should say that between distributions describe cross-country averages and within distributions describe deviations from country averages. The between distributions are based on only 15 country averages and should be interpreted cautiously. For WR, the between distribution shows wide cross-country differences, while the within distribution has a long right tail and possible high-leverage observations. For LS, the between distribution is more concentrated and slightly left-skewed, while the within distribution is centered around zero with some large positive

observations. For i, the within distribution confirms substantial within-country time variation. The bivariate graphs show that WR has a between correlation with i of 0.881 and a within correlation of -0.105, while LS has a between correlation of -0.318 and a within correlation of 0.547. These correlations are descriptive, not causal.

Prompt 6

Prompt: This is basically what I wrote before but I will structure the graphs like the within and between WR together. Check these answers are good then structure it well.

Answer: The answer is good and covers what Question 6 asks: the meaning of the between and within transformations, distribution shapes for WR, LS, and i, normality and non-normality, skewness, high-leverage observations, and the bivariate relationships with i. A stronger structure is to group the discussion by variable: first WR with its between, within, and bivariate comments; then LS; then i; then an overall conclusion. This makes it match the graph layout. The answer should emphasize that WR has the most serious right-tail problem in the within distribution, LS is less extreme, and i has substantial within-country time variation.

Prompt 7

Prompt: For Part 1, this is what I found at the end of the partial replication. Make it sound better and more academic.

Answer: The introduction was rewritten into three more academic paragraphs. The revised text explains that the replication uses the authors' dataset for 15 advanced economies from 1970 to 2019 and focuses on the short-term interest rate, real wages, and adjusted labor share. It highlights three main findings not immediately visible from the paper: the panel is slightly unbalanced because Japan starts in 1980 for WR and LS; the relationship between monetary policy and distribution depends strongly on the transformation used; and results reveal substantial country heterogeneity and sensitivity to model specification, especially once time effects are included.

Prompt 8

Prompt: This is our group's final answers for the report. Can you please check if everything is ok?

Answer: The report is mostly ready and strong, but several fixes were recommended. The database update section repeats itself and should be shortened. Table 4 should describe PCOM as an individual-invariant variable that varies over time but is common across countries, not as a 'time series invariant' variable. Table 5 should clarify that dependent variables are reported first and the remaining variables are sorted by within-variance share. Section 7 has broken formulas and should include the first-difference formula and TWFE formula explicitly. The common yearly component interpretation should be made more cautious. Question 12 should be numerically checked because the LS pre-crisis coefficient matched the full-sample TWFE result, which looked suspicious. Smaller style fixes include using one spelling of labor/labour and cleaning punctuation.

Prompt 9

Prompt: This is final code. Did we do Question 12 well?

Answer: Question 12 is conceptually good and the code structure is correct. It splits the sample into 1970-2007 and 2008-2019, estimates TWFE models for LS and WR using i, GDP, UN, and REER, and exports a summary table. The main improvement is to cluster standard errors by country using cov_type='clustered' and cluster_entity=True. After clustering, the p-values should be updated because significance may change, especially in the short post-2008 sample. The interpretation should be cautious and should avoid claiming structural change without a formal test.

Prompt 10

Prompt: I redid it to make clustered. How does Question 12 change?

Answer: After clustering standard errors by country, the coefficients remain the same but statistical significance changes substantially. None of the interest-rate coefficients in Question 12 are statistically significant at conventional levels. LS has a coefficient of -0.025 with p = 0.897 in 1970-2007 and -0.366 with p = 0.219 in 2008-2019. WR has a coefficient of -0.718 with p = 0.136 in 1970-2007 and -0.425 with p = 0.192 in 2008-2019. The conclusion should therefore be more cautious: coefficients are generally negative, especially for WR, but the estimates are imprecise and should be interpreted as exploratory robustness evidence rather than firm proof of a post-crisis change.

Prompt 11

Prompt: I need to provide the LLM prompts, so combine the ones from this chat and the attached LLM stuff from my teammates.

Answer: A combined Word and PDF file was created with the prompts from the chat and the uploaded teammate prompt files. The first version was split by source, but it needed to be revised because the professor wanted both prompts and answers in a single continuous document.

Prompt 12

Prompt: He wants LLM prompts and answers, so redo please with the new attachments and this chat. Do not split it by team member or questions. Title it 'LLM prompts for partial replication project' then go straight into the prompts and answers. Do not include source file either. Make a new PDF and Word with the attachments and this chat in a nice format including all prompts and answers. It can be arranged by question but it is all one project.

Answer: The requested output is a single combined document titled 'LLM Prompts for Partial Replication Project'. It should include prompts and answers from this chat and the newly uploaded prompt documents, without splitting by teammate or by source file. The content should be formatted as one project record with prompt-answer pairs in a clean order and delivered as both Word and PDF.

Prompt 13

I have panel data indexed by country and year. How can I estimate Between, Fixed Effects and Two-Way Fixed Effects models in Python using the variables LS, WR, i, GDP, UN and REER?

Answer 1

You can use the linearmodels.panel package. First define the panel structure using country and year as indices. Then estimate the Between, Fixed Effects and Two-Way Fixed Effects models as follows:

```
from linearmodels.panel import BetweenOLS, PanelOLS
df = df.set_index(["country", "year"])

y = df["LS"]
X = df[["i", "GDP", "UN", "REER"]]
# Between

between_model = BetweenOLS(y, X)
between_results = between_model.fit()

# Fixed Effects

fe_model = PanelOLS(
y,

X,

entity_effects=True

)

fe_results = fe_model.fit()

# Two-Way Fixed Effects

twfe_model = PanelOLS(
y,

X,

entity_effects=True,

time_effects=True

)

twfe_results = twfe_model.fit()
```

The results can then be exported to text files using Python's open() function.

Prompt 14

How can I estimate a Mundlak correlated random effects model in Python?

Answer 2

The Mundlak approach consists of augmenting a random effects model with country means of the explanatory variables.

```
from linearmodels.panel import RandomEffects
mundlak_df = df.copy()

for var in ["i", "GDP", "UN", "REER"]:
mundlak_df[f"{var}_mean"] = (
mundlak_df.groupby(level=0)[var]

.transform("mean")
```

)

```
X_mundlak = mundlak_df[
```
["i", "GDP", "UN", "REER",

"i_mean", "GDP_mean",

"UN_mean", "REER_mean"]

]

```
mundlak_model = RandomEffects(
mundlak_df["LS"],
```
X_mundlak

)

mundlak_results = mundlak_model.fit()

```
This specification allows the random effects estimator to account for correlation between country-specific
effects and explanatory variables.
```

Prompt 15

```
How can I estimate a First Differences model in Python for panel data?
```

Answer 3

The first-difference transformation removes time-invariant country effects.

```
fd_df = df[
```
["LS", "i", "GDP", "UN", "REER"]

].copy()

fd_df = fd_df.groupby(level=0).diff()

```
y_fd = fd_df["LS"]
X_fd = fd_df[
```
["i", "GDP", "UN", "REER"]

]

```
fd_model = PanelOLS(
```
y_fd,

X_fd

)

fd_results = fd_model.fit()

The resulting coefficients measure the effect of changes in explanatory variables on changes in the dependent variable.

Prompt 16

How can I compare the effect of interest rates before and after the 2008 Global Financial Crisis using a Two-Way Fixed Effects model?

Answer 4

Create two subsamples (1970–2007 and 2008–2019) and estimate the same TWFE specification on each period.

periods = {

"pre_2008": (1970, 2007),

"post_2008": (2008, 2019)

```
}

for period_name, (start_year, end_year) in periods.items():
sample = df[
(df.index.get_level_values("year") >= start_year)

&

(df.index.get_level_values("year") <= end_year)

]

model = PanelOLS(
sample["LS"],

sample[["i", "GDP", "UN", "REER"]],

entity_effects=True,

time_effects=True

)

results = model.fit()

print(period_name)

print(results.summary)
```

This approach allows the researcher to evaluate whether the relationship between monetary policy and income distribution differs before and after the Global Financial Crisis.

Prompt 17

```
I have written the Python code for Questions 10 and 12 of my panel data replication project. The code estimates
Between, Fixed Effects, Two-Way Fixed Effects, First Differences, Mundlak correlated random effects, and a split-
sample TWFE analysis before and after 2008. Can you translate this code into R and Stata while keeping the same
variable names, folder structure, and output logic?
```

Answer 5

R translation

```
library(readxl)
library(plm)
base_dir <- getwd()
data_file <- file.path(base_dir, "03_clean_data", "Dataset_MP_Impact_functional_Distribution_clean.xlsx")
tables_dir <- file.path(base_dir, "08_tables")
df <- read_excel(data_file)
df_panel <- pdata.frame(df, index = c("country", "year"))
dependent_variables <- c("LS", "WR")
explanatory_variables <- c("i", "GDP", "UN", "REER")
for (dep_var in dependent_variables) {
formula_model <- as.formula(
paste(dep_var, "~", paste(explanatory_variables, collapse = " + "))

)

# Between estimator

between_model <- plm(
formula_model,

data = df_panel,

model = "between"
)

capture.output(
summary(between_model),
```

```
file = file.path(tables_dir, paste0("q10_between_results_", dep_var, "_R.txt"))

)

# Fixed Effects / Within estimator

fe_model <- plm(
formula_model,

data = df_panel,

model = "within",
effect = "individual"

)

capture.output(
summary(fe_model),

file = file.path(tables_dir, paste0("q10_fe_results_", dep_var, "_R.txt"))

)

# Two-Way Fixed Effects

twfe_model <- plm(
formula_model,

data = df_panel,

model = "within",
effect = "twoways"

)

capture.output(
summary(twfe_model),

file = file.path(tables_dir, paste0("q10_twfe_results_", dep_var, "_R.txt"))

)

# First Differences

fd_model <- plm(
formula_model,

data = df_panel,

model = "fd"
)

capture.output(
summary(fd_model),

file = file.path(tables_dir, paste0("q10_fd_results_", dep_var, "_R.txt"))

)

# Mundlak / Correlated Random Effects

mundlak_df <- df
for (var in explanatory_variables) {
mundlak_df[[paste0(var, "_mean")]] <- ave(
mundlak_df[[var]],
mundlak_df$country,
```

```
FUN = function(x) mean(x, na.rm = TRUE)

)

}

mundlak_panel <- pdata.frame(mundlak_df, index = c("country", "year"))
mundlak_formula <- as.formula(
paste(

dep_var,

"~",

paste(

c(

explanatory_variables,

paste0(explanatory_variables, "_mean")

),

collapse = " + "

)

)

)

mundlak_model <- plm(
mundlak_formula,

data = mundlak_panel,

model = "random"
)

capture.output(
summary(mundlak_model),

file = file.path(tables_dir, paste0("q10_mundlak_results_", dep_var, "_R.txt"))

)

}
```

R code for Question 12

```
periods <- list(
pre_2008 = c(1970, 2007),

post_2008 = c(2008, 2019)

)

q12_results <- data.frame()
for (dep_var in dependent_variables) {
for (period_name in names(periods)) {
start_year <- periods[[period_name]][1]
end_year <- periods[[period_name]][2]
sample_df <- subset(
df,
```

```
year >= start_year & year <= end_year

)

sample_panel <- pdata.frame(sample_df, index = c("country", "year"))
formula_model <- as.formula(
paste(dep_var, "~ i + GDP + UN + REER")

)

twfe_model <- plm(
formula_model,

data = sample_panel,

model = "within",
effect = "twoways"

)

model_summary <- summary(twfe_model)
capture.output(
model_summary,

file = file.path(

tables_dir,

paste0("q12_twfe_", dep_var, "_", period_name, "_R.txt")
)

)

coef_i <- model_summary$coefficients["i", "Estimate"]
pval_i <- model_summary$coefficients["i", "Pr(>|t|)"]
q12_results <- rbind(
q12_results,
data.frame(

dependent_variable = dep_var,

period = period_name,

start_year = start_year,

end_year = end_year,

coefficient_i = coef_i,

p_value_i = pval_i

)

)

}

}

write.csv(

q12_results,
file.path(tables_dir, "q12_split_sample_twfe_summary_R.csv"),
row.names = FALSE

)
```

Translation note for R

In R, the plm package is used instead of Python’s linearmodels. The Between, Within, Two-Way Fixed Effects and First Differences models are directly available in plm. For the Mundlak model, country-level means are created manually with ave() and then included in a random effects model.

Prompt 18

I have already created between, within, first-difference and two-way fixed effects transformed variables in Python. How can I generate summary statistics (mean, median, standard deviation, quartiles, minimum and maximum) for each transformation and export the results to excel?
Summary statistics for transformations
q8_summary_rows = []
Q8_VARS = ["WR", "LS", "i"]

```
for var in Q8_VARS:
    transformations = {
        "Between": q8_between[f"{var}_between"],
        "One-way within": q8_within[f"{var}_within"],
        "First differences": q8_fd[f"d_{var}"],
        "Two-way fixed effects": q8_twfe[f"twfe_{var}"],
    }
    for transformation_name, series in transformations.items():
        values = series.replace([np.inf, -np.inf], np.nan).dropna()
        std = values.std(ddof=1)
        q8_summary_rows.append({
            "Variable": var,
            "Transformation": transformation_name,
            "N": len(values),
            "Mean": values.mean(),
            "Median": values.median(),
            "Standard deviation": std,
            "Standard error": std / np.sqrt(len(values)),
            "Q1": values.quantile(0.25),
            "Q3": values.quantile(0.75),
            "Standardized min": (values.min() - values.mean()) / std,
            "Standardized max": (values.max() - values.mean()) / std,
        })
q8_summary = pd.DataFrame(q8_summary_rows)
q8_summary.to_excel(
    TABLES_DIR / "q8_transformation_summary_statistics.xlsx",
    index=False
)
```

This code creates one table comparing the four transformations for WR, LS, and i. The between transformation uses country averages, the one-way within transformation removes country means, first differences measure year-to-year changes, and TWFE removes both country and year effects.

Prompt 19

I have panel data with countries and years. How can I create boxplots by country for first-difference and two-way fixed effects transformed variables in python using pandas and matplotlib?
Boxplots by country

```
def save_q8_boxplot_by_country(data, value_col, title, filename):
    plot_data = data[["country", value_col]].replace([np.inf, -np.inf], np.nan).dropna()
    if plot_data.empty:
        return
    country_variance = (
        plot_data.groupby("country")[value_col]
        .var()
        .sort_values(ascending=True)
    )
    ordered_countries = country_variance.index.tolist()
    values_by_country = [
        plot_data.loc[plot_data["country"] == country, value_col]
        for country in ordered_countries
    ]
    plt.figure(figsize=(11, 6))
    plt.boxplot(values_by_country, labels=ordered_countries, showfliers=True)
    plt.xticks(rotation=45, ha="right")
    plt.axhline(0, linestyle=":")
    plt.title(title)
    plt.xlabel("Country")
    plt.ylabel(value_col)
    plt.tight_layout()
    plt.savefig(FIGURES_DIR / filename, dpi=300)
    plt.close()
for var in ["WR", "LS", "i"]:
    save_q8_boxplot_by_country(
        q8_within,
        f"{var}_within",
        f"One-way within distribution by country: {var}",
        f"q8_boxplot_within_by_country_{var}.png"
    )
    save_q8_boxplot_by_country(
        q8_fd,
```

```
        f"d_{var}",
        f"First differences distribution by country: {var}",
        f"q8_boxplot_fd_by_country_{var}.png"
    )
    save_q8_boxplot_by_country(
        q8_twfe,
        f"twfe_{var}",
        f"Two-way fixed effects distribution by country: {var}",
        f"q8_boxplot_twfe_by_country_{var}.png"
    )
```

For the between transformation, use one overall boxplot because each country only has one country-average value:

```
def save_between_boxplot(series, title, filename):
    values = pd.Series(series).replace([np.inf, -np.inf], np.nan).dropna()
    plt.figure(figsize=(6, 5))
    plt.boxplot(values, showfliers=True)
    plt.xticks([1], ["All countries"])
    plt.title(title)
    plt.ylabel("Country averages")
    plt.tight_layout()
    plt.savefig(FIGURES_DIR / filename, dpi=300)
    plt.close()
for var in ["WR", "LS", "i"]:
    save_between_boxplot(
        q8_between[f"{var}_between"],
        f"Between distribution across countries: {var}",
        f"q8_boxplot_between_all_countries_{var}.png"
    )
```

Prompt 20

How can I verify that first-difference transformations are calculated correctly?

Verify first differences

To verify first differences, check that the first observation of each country is missing and that differences are only computed within the same country.

```
q7_data = df.copy()
q7_data = q7_data.sort_values(["country", "year"])
q7_data["year_gap"] = q7_data.groupby("country")["year"].diff()
consecutive_year = q7_data["year_gap"].eq(1)
for var in ["WR", "LS", "i"]:
    q7_data[f"d_{var}"] = (
        q7_data.groupby("country")[var].diff()
    ).where(consecutive_year)
```

Then create a boundary check table:

```
country_start_rows = q7_data.index[
    q7_data["country"].ne(q7_data["country"].shift())
```

].tolist()

rows_to_check = []

```
for row in country_start_rows:
    rows_to_check.extend([row, row + 1])
```

rows_to_check = sorted(set(

```
    row for row in rows_to_check if 0 <= row < len(q7_data)
```

))

```
first_difference_check = q7_data.loc[
    rows_to_check,
    ["country", "year", "WR", "d_WR", "LS", "d_LS", "i", "d_i"]
```

]

first_difference_check.to_excel(

```
    TABLES_DIR / "first_difference_boundary_check.xlsx",
    index=False
```

)

```
Expected result: for each country’s first year, d_WR, d_LS, and d_i should be missing. This confirms that the
code did not mistakenly subtract the previous country’s final observation.
```

Prompt 21

I have a Python script for a panel data replication project that I need translated into R and Stata. I'll paste the code below. For both translations, please keep the exact same structure, variable names, output filenames and folder layout as the original. Don't simplify or skip any section.
For R, use base graphics throughout, the plm package for panel estimators.
In both versions, add short comments where the translation required a different approach, just a sentence or two explaining what changed and why, written plainly.

Here is the script:

Answer:

R code:

```
# ============================================================

# To run: open a terminal and type  Rscript panel_replication.R

# In RStudio you can also just click Source or press Ctrl+Shift+S.

#

# Packages needed (run once):

#   install.packages(c("readxl", "writexl", "dplyr", "tidyr", "plm"))

# ============================================================

# ============================================================

# 0. PACKAGES

# ============================================================

# If any package is missing this will tell you which one and stop.

required_pkgs <- c("readxl", "writexl", "dplyr", "tidyr", "plm")
missing_pkgs  <- required_pkgs[!sapply(required_pkgs, requireNamespace, quietly = TRUE)]
if (length(missing_pkgs) > 0) {
  stop(paste("Please install missing packages:\n",
             "install.packages(c(",
             paste0('"', missing_pkgs, '"', collapse = ", "),
             "))"))
}

library(readxl)
library(writexl)
library(dplyr)
library(tidyr)
library(plm)
# ============================================================

# 1. PATHS

# ============================================================

# Python finds the script location automatically via __file__. R has no

# equivalent that works everywhere, so we use commandArgs() which works

# from the terminal and VS Code. If you run interactively in RStudio and
# the paths come out wrong, just replace SCRIPT_DIR with the hardcoded

# path to wherever you saved this file.

args        <- commandArgs(trailingOnly = FALSE)
script_arg  <- args[startsWith(args, "--file=")]
if (length(script_arg) > 0) {
```

```
  # Running via Rscript in a terminal
  SCRIPT_DIR <- dirname(normalizePath(sub("--file=", "", script_arg[1])))
} else {

  # Running interactively (RStudio / VS Code R terminal)
  # Falls back to working directory — make sure setwd() points to your
  # script folder, or replace the line below with a hardcoded path.
  SCRIPT_DIR <- getwd()
  message("NOTE: Could not detect script path automatically.",
          " Using working directory: ", SCRIPT_DIR,
          "\nIf outputs go to the wrong place, run setwd() first",
          " or hardcode SCRIPT_DIR.")
}

BASE_DIR    <- dirname(SCRIPT_DIR)    # one level up from the script folder
DATA_DIR    <- file.path(BASE_DIR, "02_original_data")
CLEAN_DIR   <- file.path(BASE_DIR, "03_clean_data")
TABLES_DIR  <- file.path(BASE_DIR, "08_tables")
FIGURES_DIR <- file.path(BASE_DIR, "07_figures")
for (d in c(CLEAN_DIR, TABLES_DIR, FIGURES_DIR)) {
  dir.create(d, recursive = TRUE, showWarnings = FALSE)
}

DATA_FILE       <- file.path(DATA_DIR,  "Dataset_MP_Impact_functional_Distribution.xlsx")
CLEAN_DATA_FILE <- file.path(CLEAN_DIR, "Dataset_MP_Impact_functional_Distribution_clean.xlsx")
# ============================================================

# 2. VARIABLE DEFINITIONS

# ============================================================

DEPENDENT_VARIABLES <- c("WR", "LS")
KEY_X               <- "i"
ALL_VARIABLES <- c(
  "i", "P", "W", "WR", "GDP", "LS", "PCOM", "UN",
  "SHORTUN", "LONGUN", "LF", "REER", "SH"
)

VARIABLE_LABELS <- c(
  i       = "Short-term interest rate, i",
  P       = "GDP deflator / price level, P",
  W       = "Nominal compensation per employee, W",
  WR      = "Dependent variable: Real wages, WR",
  GDP     = "Real GDP, GDP",
  LS      = "Dependent variable: Labor share, LS",
  PCOM    = "Energy commodity price index, PCOM",
  UN      = "Unemployment rate, UN",
  SHORTUN = "Short-term unemployment, SHORTUN",
  LONGUN  = "Long-term unemployment, LONGUN",
  LF      = "Labor force, LF",
  REER    = "Real effective exchange rate, REER",
  SH      = "Shadow interest rate, SH"
)

# ============================================================

# 3. LOAD AND CLEAN DATA

# ============================================================

if (!file.exists(DATA_FILE)) {
  stop(paste("Could not find the dataset:\n", DATA_FILE,
             "\nCheck that the Excel file is in 02_original_data/"))
}

df        <- read_excel(DATA_FILE)
names(df) <- trimws(names(df))
df        <- df %>% arrange(country, year)
countries <- sort(unique(na.omit(df$country)))
years     <- sort(unique(na.omit(df$year)))
write_xlsx(df, CLEAN_DATA_FILE)
# Python does this with a list comprehension; intersect() is the R equivalent.

ALL_VARIABLES <- intersect(ALL_VARIABLES, names(df))
```

```
cat("Dataset loaded successfully.\n")

cat("Rows x cols:", nrow(df), "x", ncol(df), "\n")

cat("Countries:", length(countries), "\n")

cat("Years:", min(years), "to", max(years), "\n")

# ============================================================

# 4. HELPER FUNCTIONS

# ============================================================

# ------ consecutive_blocks ------

# Same logic as the Python version, just a regular for-loop instead of

# a generator. Output is identical.

consecutive_blocks <- function(year_list) {
  year_list <- sort(year_list)
  if (length(year_list) == 0) return(list())
  blocks <- list()
  start  <- year_list[1]
  prev   <- year_list[1]
  for (yr in year_list[-1]) {
    if (yr == prev + 1) {
      prev <- yr
    } else {
      blocks <- c(blocks, list(c(start, prev, prev - start + 1)))
      start  <- yr
      prev   <- yr
    }
  }
  blocks <- c(blocks, list(c(start, prev, prev - start + 1)))
  return(blocks)
}

# ------ normal_density_vals ------

normal_density_vals <- function(x, mu, sigma) {
  if (is.na(sigma) || sigma <= 0) return(rep(0, length(x)))
  (1 / (sigma * sqrt(2 * pi))) * exp(-0.5 * ((x - mu) / sigma)^2)
}

# ------ skewness / kurtosis ------

# Base R does not have these. We compute them manually using the same

# formulas as pandas so the numbers match exactly.

skewness_r <- function(x) {
  x <- na.omit(x)
  n <- length(x)
  if (n < 3) return(NA_real_)
  mu <- mean(x); s <- sd(x)
  if (s == 0) return(NA_real_)
  (sum((x - mu)^3) / n) / s^3
}

kurtosis_r <- function(x) {
  x <- na.omit(x)
  n <- length(x)
  if (n < 4) return(NA_real_)
  mu <- mean(x); s <- sd(x)
  if (s == 0) return(NA_real_)
  (sum((x - mu)^4) / n) / s^4 - 3   # excess kurtosis, same as pandas
}

# ------ plot_hist_kde_normal ------
```

```
# Base R needs a separate density() call and manual x-range for the
# normal curve overlay. We use base graphics to avoid extra dependencies.

plot_hist_kde_normal <- function(values, title, xlabel, save_path) {
  values <- na.omit(as.numeric(values))
  if (length(values) < 3) {
    cat("Not enough observations to plot:", title, "\n")
    return(invisible(NULL))
  }
  mu    <- mean(values)
  sigma <- sd(values)
  x_seq <- seq(min(values), max(values), length.out = 300)
  h     <- density(values, adjust = 1)   # adjust=1 matches scipy default
  png(save_path, width = 800, height = 500, res = 150)
  hist(values, freq = FALSE, breaks = 15,
       col  = rgb(0.2, 0.4, 0.8, 0.45),
       main = title, xlab = xlabel, ylab = "Density",
       border = "white")
  lines(h$x, h$y, col = "steelblue", lwd = 2)
  lines(x_seq, normal_density_vals(x_seq, mu, sigma), col = "red", lwd = 2)
  legend("topright",
         legend = c("Histogram", "Kernel density", "Normal distribution"),
         fill   = c(rgb(0.2, 0.4, 0.8, 0.45), NA, NA),
         border = c("white", NA, NA),
         lty    = c(NA, 1, 1),
         col    = c(NA, "steelblue", "red"),
         lwd    = 2, bty = "n")
  dev.off()
}

# ------ scatter_with_regression ------

scatter_with_regression <- function(data, x_col, y_col, title, save_path) {
  temp <- na.omit(data[, c(x_col, y_col)])
  if (nrow(temp) < 3) {
    cat("Not enough observations to plot:", title, "\n")
    return(invisible(NULL))
  }
  corr  <- cor(temp[[x_col]], temp[[y_col]])
  fit   <- lm(as.formula(paste(y_col, "~", x_col)), data = temp)
  x_seq <- seq(min(temp[[x_col]]), max(temp[[x_col]]), length.out = 200)
  y_hat <- coef(fit)[1] + coef(fit)[2] * x_seq
  png(save_path, width = 800, height = 500, res = 150)
  plot(temp[[x_col]], temp[[y_col]],
       pch  = 16, col  = rgb(0, 0, 0, 0.55),
       main = paste0(title, "\nCorrelation = ", round(corr, 3)),
       xlab = x_col, ylab = y_col)
  lines(x_seq, y_hat, col = "blue", lwd = 2)
  legend("topright", legend = "Linear fit", col = "blue", lty = 1, lwd = 2)
  grid()
  dev.off()
}

# ============================================================

# 5. SAMPLE SELECTION (Questions 1–3)

# ============================================================

sample_rows <- lapply(countries, function(ctry) {
  row <- list(Country = ctry)
  for (dep in DEPENDENT_VARIABLES) {
    avail      <- df$year[df$country == ctry & !is.na(df[[dep]])]
    blocks     <- consecutive_blocks(avail)
    max_consec <- if (length(blocks) > 0) max(sapply(blocks, `[`, 3)) else 0L
    row[[paste0(dep, ": number of non-missing observations")]]   <- length(avail)
    row[[paste0(dep, ": first available year")]]                  <- if (length(avail)) min(avail) else NA_real_
    row[[paste0(dep, ": last available year")]]                   <- if (length(avail)) max(avail) else NA_real_
    row[[paste0(dep, ": maximum consecutive observations")]]      <- max_consec
    row[[paste0(dep, ": kept in sample")]]                        <- max_consec >= 3
  }
  row
})

sample_selection_table <- bind_rows(sample_rows)
write_xlsx(sample_selection_table, file.path(TABLES_DIR, "sample_selection.xlsx"))
kept_all     <- sample_selection_table %>%
  filter(`WR: kept in sample` == TRUE, `LS: kept in sample` == TRUE)
```

```
excluded_all <- sample_selection_table %>%
  filter(`WR: kept in sample` == FALSE | `LS: kept in sample` == FALSE)
sample_selection_summary <- data.frame(
  Item  = c("Total number of countries", "Number of excluded countries",
            "Number of kept countries",  "Excluded countries", "Kept countries"),
  Value = c(length(countries), nrow(excluded_all), nrow(kept_all),
            if (nrow(excluded_all) > 0) paste(excluded_all$Country, collapse = ", ") else "None",
            paste(kept_all$Country, collapse = ", ")),
  stringsAsFactors = FALSE
)

write_xlsx(sample_selection_summary, file.path(TABLES_DIR, "sample_selection_summary.xlsx"))
# ============================================================

# 6. PANEL STRUCTURE: COUNTRIES PER YEAR AND HOLES

# ============================================================

dep <- "WR"
countries_per_year <- df %>%
  filter(!is.na(.data[[dep]])) %>%
  group_by(year) %>%
  summarise(`Number of countries` = n_distinct(country), .groups = "drop")
write_xlsx(countries_per_year, file.path(TABLES_DIR, "number_of_countries_per_year.xlsx"))
png(file.path(FIGURES_DIR, "number_of_countries_per_year.png"),
    width = 1000, height = 500, res = 150)
plot(countries_per_year$year, countries_per_year$`Number of countries`,

     type = "o", pch = 16,
     main = "Number of countries observed per year",
     xlab = "Year", ylab = "Number of countries")
grid()

dev.off()

write_xlsx(
  data.frame(Date = c("1970-1979", "1980-2019"), N = c(14L, 15L)),
  file.path(TABLES_DIR, "compact_number_of_individuals_per_date.xlsx")
)

observations_by_country <- df %>%
  filter(!is.na(.data[[dep]])) %>%
  group_by(country) %>%
  summarise(`Number of observations` = n_distinct(year), .groups = "drop") %>%
  arrange(desc(`Number of observations`))
write_xlsx(observations_by_country, file.path(TABLES_DIR, "observations_by_country.xlsx"))
same_number_of_observations <- observations_by_country %>%
  group_by(`Number of observations`) %>%
  summarise(`Number of countries` = n(), .groups = "drop") %>%
  arrange(desc(`Number of observations`))
write_xlsx(same_number_of_observations, file.path(TABLES_DIR, "same_number_of_observations.xlsx"))
holes_rows <- lapply(countries, function(ctry) {
  avail   <- sort(df$year[df$country == ctry & !is.na(df[[dep]])])
  if (length(avail) == 0) return(NULL)
  missing <- sort(setdiff(seq(min(avail), max(avail)), avail))
  list(
    Country                           = ctry,
    `First available year`            = min(avail),
    `Last available year`             = max(avail),
    `Number of observations`          = length(avail),
    `Has holes`                       = length(missing) > 0,
    `Number of missing years inside span` = length(missing),
    `Missing years inside span`       = if (length(missing)) paste(missing, collapse = ", ") else "None"
  )
})

holes_table <- bind_rows(Filter(Negate(is.null), holes_rows))
write_xlsx(holes_table, file.path(TABLES_DIR, "holes_inside_panel.xlsx"))
write_xlsx(
  data.frame(
    Item  = c("Number of countries with holes", "Proportion of countries with holes"),
    Value = c(sum(holes_table$`Has holes`), mean(holes_table$`Has holes`))
  ),
  file.path(TABLES_DIR, "holes_summary.xlsx")
)
```

```
# ============================================================

# 7. WITHIN / BETWEEN VARIANCE DECOMPOSITION

# ============================================================

# Same logic as the Python loop — lapply over variable names, bind rows at the end.

variance_rows <- lapply(ALL_VARIABLES, function(var) {
  temp <- df %>% select(country, year, all_of(var)) %>% na.omit()
  if (nrow(temp) == 0) return(NULL)
  overall_var  <- var(temp[[var]])
  country_avg  <- tapply(temp[[var]], temp$country, mean)
  between_var  <- var(country_avg)
  temp$cmean   <- ave(temp[[var]], temp$country, FUN = mean)
  within_var   <- var(temp[[var]] - temp$cmean)
  within_share <- if (!is.na(overall_var) && overall_var != 0) within_var / overall_var else NA_real_
  list(
    Variable            = var,
    `Variable in words` = ifelse(var %in% names(VARIABLE_LABELS), VARIABLE_LABELS[var], var),
    N                   = length(unique(temp$country)),
    NT                  = nrow(temp),
    `NT/N`              = nrow(temp) / length(unique(temp$country)),
    `Overall variance`  = overall_var,
    `Between variance`  = between_var,
    `Within variance`   = within_var,
    `Within share`      = within_share,
    `Within share (%)`  = within_share * 100
  )
})

variance_table <- bind_rows(Filter(Negate(is.null), variance_rows)) %>%
  arrange(desc(`Within share`))
write_xlsx(variance_table, file.path(TABLES_DIR, "within_between_variance_all_variables.xlsx"))

# Split into three categories

two_index_vars            <- character(0)
time_invariant_vars       <- character(0)
individual_invariant_vars <- character(0)
for (i in seq_len(nrow(variance_table))) {
  var <- variance_table$Variable[i]
  ws  <- variance_table$`Within share`[i]
  if (is.na(ws))              next
  if (abs(ws) < 1e-6)         time_invariant_vars       <- c(time_invariant_vars,       var)
  else if (abs(ws - 1) < 1e-6) individual_invariant_vars <- c(individual_invariant_vars, var)
  else                         two_index_vars            <- c(two_index_vars,            var)
}

write_xlsx(
  data.frame(
    `List of variables varying with two indices (time and individuals)` =
      if (length(two_index_vars)) paste(two_index_vars, collapse = ", ") else "None",
    `List of time-invariant variables` =
      if (length(time_invariant_vars)) paste(time_invariant_vars, collapse = ", ") else "None",
    `List of individual-invariant variables` =
      if (length(individual_invariant_vars)) paste(individual_invariant_vars, collapse = ", ") else "None",
    `Number K`  = length(two_index_vars),
    `Number K1` = length(time_invariant_vars),
    `Number K2` = length(individual_invariant_vars),
    check.names = FALSE
  ),
  file.path(TABLES_DIR, "variable_categories.xlsx")
)

# Table 5: time-varying variables only

time_varying  <- variance_table %>% filter(`Within share` > 0, `Within share` < 1)
dep_rows      <- time_varying %>% filter(Variable %in% DEPENDENT_VARIABLES) %>%
  mutate(order = match(Variable, c("WR", "LS"))) %>% arrange(order) %>% select(-order)
other_rows    <- time_varying %>% filter(!Variable %in% DEPENDENT_VARIABLES) %>%
  arrange(desc(`Within share`))
time_varying_for_word <- bind_rows(dep_rows, other_rows) %>%
  select(`Variable in words`, N, NT, `NT/N`,
         `Overall variance`, `Between variance`, `Within variance`, `Within share (%)`) %>%
  mutate(across(c(`NT/N`, `Overall variance`, `Between variance`,
                  `Within variance`, `Within share (%)`), ~round(.x, 2)))
write_xlsx(time_varying_for_word, file.path(TABLES_DIR, "time_varying_variables_for_word.xlsx"))
```

```
# ============================================================

# 8. BETWEEN AND ONE-WAY WITHIN DECOMPOSITION

# ============================================================

bw_data <- df %>% select(country, year, WR, LS, i)
for (var in c("WR", "LS", "i")) {
  bw_data[[paste0(var, "_between")]] <- ave(bw_data[[var]], bw_data$country, FUN = function(x) mean(x, na.rm =
TRUE))
  bw_data[[paste0(var, "_within")]]  <- bw_data[[var]] - bw_data[[paste0(var, "_between")]]
}

write_xlsx(bw_data, file.path(TABLES_DIR, "between_within_variables.xlsx"))
# Distribution plots and descriptive statistics

bw_stats_rows <- list()
for (var in c("WR", "LS", "i")) {
  between_vals <- na.omit(tapply(df[[var]], df$country, mean))
  within_vals  <- na.omit(bw_data[[paste0(var, "_within")]])
  plot_hist_kde_normal(between_vals,
    title     = paste("Between distribution of", var),
    xlabel    = paste(var, "country average"),
    save_path = file.path(FIGURES_DIR, paste0("between_distribution_", var, ".png")))
  plot_hist_kde_normal(within_vals,
    title     = paste("One-way within distribution of", var),
    xlabel    = paste(var, "minus country average"),
    save_path = file.path(FIGURES_DIR, paste0("within_distribution_", var, ".png")))
  for (pair in list(list("Between", between_vals), list("One-way within", within_vals))) {
    vals <- pair[[2]]
    bw_stats_rows[[length(bw_stats_rows) + 1]] <- list(
      Variable            = var,
      Transformation      = pair[[1]],
      Observations        = length(vals),
      Mean                = mean(vals),
      Median              = median(vals),
      `Standard deviation` = sd(vals),
      Minimum             = min(vals),
      Maximum             = max(vals),
      Skewness            = skewness_r(vals),
      Kurtosis            = kurtosis_r(vals)
    )
  }
}

write_xlsx(bind_rows(bw_stats_rows),
           file.path(TABLES_DIR, "between_within_descriptive_statistics.xlsx"))
# Bivariate: between / within vs i

between_country <- df %>%
  group_by(country) %>%
  summarise(WR_between = mean(WR, na.rm = TRUE),
            LS_between = mean(LS, na.rm = TRUE),
            i_between  = mean(i,  na.rm = TRUE),
            .groups = "drop")
corr_rows <- list()
for (y in c("WR", "LS")) {
  corr_b <- cor(between_country[[paste0(y, "_between")]],
                between_country$i_between, use = "complete.obs")
  temp_w <- na.omit(bw_data[, c(paste0(y, "_within"), "i_within")])
  corr_w <- cor(temp_w[[1]], temp_w[[2]])
  corr_rows <- c(corr_rows,
    list(list(`Dependent variable` = y, Transformation = "Between",        `Correlation with i` = corr_b)),
    list(list(`Dependent variable` = y, Transformation = "One-way within", `Correlation with i` = corr_w)))
  scatter_with_regression(between_country, "i_between", paste0(y, "_between"),
    paste("Between relationship between i and", y),
    file.path(FIGURES_DIR, paste0("between_scatter_i_", y, ".png")))
  scatter_with_regression(bw_data, "i_within", paste0(y, "_within"),
    paste("One-way within relationship between i and", y),
    file.path(FIGURES_DIR, paste0("within_scatter_i_", y, ".png")))
}

write_xlsx(bind_rows(corr_rows),
           file.path(TABLES_DIR, "between_within_correlations_with_i.xlsx"))
# ============================================================

# 9. QUESTION 7 — FIRST DIFFERENCES AND TWO-WAY FIXED EFFECTS
```

```
# ============================================================

ID_COL      <- "country"
TIME_COL    <- "year"
KEY_VARS_Q7 <- c("WR", "LS", "i")
Y_VARS_Q7   <- c("WR", "LS")
X_VAR_Q7    <- "i"
# ------ save_distribution_q7 ------

save_distribution_q7 <- function(series, title, filename, xlabel) {
  values <- na.omit(series[is.finite(series)])
  if (length(values) < 3) {
    cat("Not enough observations to plot:", title, "\n")
    return(invisible(NULL))
  }
  mu    <- mean(values)
  sigma <- sd(values)
  x_seq <- seq(min(values), max(values), length.out = 250)
  h     <- density(values, adjust = 1)
  png(file.path(FIGURES_DIR, filename), width = 800, height = 500, res = 150)
  hist(values, freq = FALSE, breaks = 30,
       col = rgb(0.2, 0.4, 0.8, 0.45), border = "white",
       main = title, xlab = xlabel, ylab = "Density")
  if (sigma > 0) {
    lines(x_seq, normal_density_vals(x_seq, mu, sigma), col = "red", lwd = 2)
    lines(h$x, h$y, col = "steelblue", lwd = 2)
  }
  abline(v = mu, lty = 3, col = "black")
  legend("topright",
         legend = c("Normal distribution", "Kernel density", "Mean"),
         col    = c("red", "steelblue", "black"),
         lty    = c(1, 1, 3), lwd = 2, bty = "n")
  dev.off()
}

# ------ save_scatter_with_marginals_q7 ------

# R's layout() does the same thing as matplotlib's GridSpec but requires

# manual positioning. A bit more code but the output looks the same.

save_scatter_with_marginals_q7 <- function(data, x_col, y_col, title, filename) {
  pd <- na.omit(data[, c(x_col, y_col)])
  pd <- pd[is.finite(pd[[x_col]]) & is.finite(pd[[y_col]]), ]
  if (nrow(pd) < 3) {
    cat("Not enough observations to plot:", title, "\n")
    return(invisible(NULL))
  }
  x    <- pd[[x_col]]; y <- pd[[y_col]]
  corr <- cor(x, y)
  fit  <- lm(y ~ x)
  x_line <- seq(min(x), max(x), length.out = 100)
  y_line <- coef(fit)[1] + coef(fit)[2] * x_line
  png(file.path(FIGURES_DIR, filename), width = 800, height = 800, res = 150)
  # layout: [top-hist | blank] / [scatter | right-hist]
  layout(matrix(c(2, 0, 1, 3), 2, 2, byrow = TRUE),
         widths = c(3, 1), heights = c(1, 3))
  par(mar = c(5, 4, 1, 1))
  plot(x, y, pch = 16, col = rgb(0, 0, 0, 0.65), xlab = x_col, ylab = y_col)
  lines(x_line, y_line, col = "blue", lwd = 2)
  legend("topright", legend = paste0("Linear fit, corr = ", round(corr, 3)),
         col = "blue", lty = 1, lwd = 2)
  par(mar = c(0, 4, 3, 1))
  hist(x, breaks = 25, main = title, xaxt = "n", ylab = "", col = "grey80", border = "white")
  par(mar = c(5, 0, 1, 2))
  hist(y, breaks = 25, horiz = TRUE, main = "", yaxt = "n", xlab = "", col = "grey80", border = "white")
  dev.off()
  layout(1); par(mar = c(5.1, 4.1, 4.1, 2.1))  # reset layout
}

# ------ twfe_transform ------

twfe_transform <- function(data, variable) {
  cmean  <- ave(data[[variable]], data[[ID_COL]],  FUN = function(x) mean(x, na.rm = TRUE))
  ymean  <- ave(data[[variable]], data[[TIME_COL]], FUN = function(x) mean(x, na.rm = TRUE))
  gmean  <- mean(data[[variable]], na.rm = TRUE)
  data[[variable]] - cmean - ymean + gmean
```

```
}

# ------ save_country_boxplot_q7 ------

save_country_boxplot_q7 <- function(data, variable, filename, ylabel) {
  pd <- na.omit(data[, c(ID_COL, variable)])
  pd <- pd[is.finite(pd[[variable]]), ]
  if (nrow(pd) == 0) return(invisible(NULL))
  country_var <- tapply(pd[[variable]], pd[[ID_COL]], var, na.rm = TRUE)
  ordered     <- names(sort(country_var))
  vals_list   <- lapply(ordered, function(ctry) pd[[variable]][pd[[ID_COL]] == ctry])
  png(file.path(FIGURES_DIR, filename), width = 1100, height = 600, res = 150)
  boxplot(vals_list, names = ordered, las = 2,
          main = paste("Two-way fixed effects boxplot of", variable, "by country"),
          xlab = "Country", ylab = ylabel)
  abline(h = 0, lty = 3)
  dev.off()
}

# ------ country_correlation_table_q7 ------

country_correlation_table_q7 <- function(data, x_col, y_col) {
  rows <- lapply(split(data, data[[ID_COL]]), function(cd) {
    s     <- na.omit(cd[, c(x_col, y_col)])
    s     <- s[is.finite(s[[x_col]]) & is.finite(s[[y_col]]), ]
    n_obs <- nrow(s)
    if (n_obs < 3 || length(unique(s[[x_col]])) <= 1) {
      return(list(country = cd[[ID_COL]][1], N = n_obs,
                  correlation = NA_real_, std_y = NA_real_,
                  std_x = NA_real_, simple_slope = NA_real_))
    }
    corr  <- cor(s[[x_col]], s[[y_col]])
    std_x <- sd(s[[x_col]]); std_y <- sd(s[[y_col]])
    list(country = cd[[ID_COL]][1], N = n_obs, correlation = corr,
         std_y = std_y, std_x = std_x,
         simple_slope = if (std_x != 0) corr * std_y / std_x else NA_real_)
  })
  bind_rows(rows) %>% arrange(desc(correlation))
}

# ---- First differences ----

# First differences using lag() inside group_by(). The data needs to be

# sorted by year first or the lag will pick up the wrong row. The year_gap

# check mirrors what Python does with the year_gap == 1 guard.

q7_data <- df %>% arrange(country, year) %>%
  group_by(country) %>%
  mutate(year_gap = year - lag(year),
         across(all_of(KEY_VARS_Q7),
                ~ifelse(!is.na(lag(.)) & (year - lag(year)) == 1, . - lag(.), NA_real_),
                .names = "d_{.col}")) %>%
  ungroup() %>%
  as.data.frame()
# Boundary check: verify no cross-country differences

country_starts <- which(q7_data$country != c(NA, head(q7_data$country, -1)))
rows_to_check  <- sort(unique(c(country_starts - 1, country_starts, country_starts + 1)))
rows_to_check  <- rows_to_check[rows_to_check >= 1 & rows_to_check <= nrow(q7_data)]
rows_to_check  <- head(rows_to_check, 12)
write_xlsx(q7_data[rows_to_check, c("country","year","WR","d_WR","LS","d_LS","i","d_i")],
           file.path(TABLES_DIR, "q7_first_difference_boundary_check.xlsx"))
# FD descriptive stats

fd_vars_q7 <- paste0("d_", KEY_VARS_Q7)
fd_desc <- do.call(rbind, lapply(fd_vars_q7, function(v) {
  x <- na.omit(q7_data[[v]])
  data.frame(Variable = v, N = length(x), Mean = mean(x), SD = sd(x),
             Min = min(x), Max = max(x))
}))

write_xlsx(fd_desc, file.path(TABLES_DIR, "q7_first_difference_descriptive_statistics.xlsx"))
# FD correlations
```

```
fd_corr <- cor(q7_data[fd_vars_q7], use = "pairwise.complete.obs")
write_xlsx(as.data.frame(fd_corr), file.path(TABLES_DIR, "q7_first_difference_correlations.xlsx"))
for (var in KEY_VARS_Q7) {
  save_distribution_q7(q7_data[[paste0("d_", var)]],
    title = paste("First difference distribution of", var),
    filename = paste0("q7_fd_distribution_", var, ".png"),
    xlabel = paste("First difference of", var))
}

for (y_var in Y_VARS_Q7) {
  save_scatter_with_marginals_q7(q7_data,
    x_col = paste0("d_", X_VAR_Q7), y_col = paste0("d_", y_var),
    title = paste0("First differences: d_", X_VAR_Q7, " and d_", y_var),
    filename = paste0("q7_fd_scatter_d_", X_VAR_Q7, "_d_", y_var, ".png"))
}
```

---- Balanced panel TWFE ----

```
complete_data <- q7_data %>% filter(if_all(all_of(KEY_VARS_Q7), ~!is.na(.)))
obs_by_country     <- complete_data %>% group_by(country) %>%
  summarise(T = n_distinct(year), .groups = "drop")
max_t              <- max(obs_by_country$T)
balanced_countries <- obs_by_country$country[obs_by_country$T == max_t]
twfe_data    <- complete_data %>% filter(country %in% balanced_countries)
year_counts  <- twfe_data %>% group_by(year) %>%
  summarise(n = n_distinct(country), .groups = "drop")
common_years <- year_counts$year[year_counts$n == length(balanced_countries)]
twfe_data    <- twfe_data %>% filter(year %in% common_years) %>%
  arrange(country, year) %>% as.data.frame()
write_xlsx(data.frame(
  item  = c("Number of countries","Number of years","First year","Last year","Number of observations"),
  value = c(length(unique(twfe_data$country)), length(unique(twfe_data$year)),
            min(twfe_data$year), max(twfe_data$year), nrow(twfe_data))),
  file.path(TABLES_DIR, "q7_twfe_balanced_sample_summary.xlsx"))
write_xlsx(data.frame(country = balanced_countries),
           file.path(TABLES_DIR, "q7_twfe_balanced_countries.xlsx"))
for (var in KEY_VARS_Q7) twfe_data[[paste0("twfe_", var)]] <- twfe_transform(twfe_data, var)
twfe_vars_q7 <- paste0("twfe_", KEY_VARS_Q7)
twfe_desc <- do.call(rbind, lapply(twfe_vars_q7, function(v) {
  x <- na.omit(twfe_data[[v]])
  data.frame(Variable = v, N = length(x), Mean = mean(x), SD = sd(x),
             Min = min(x), Max = max(x))
}))

write_xlsx(twfe_desc, file.path(TABLES_DIR, "q7_twfe_descriptive_statistics.xlsx"))
write_xlsx(as.data.frame(cor(twfe_data[twfe_vars_q7], use = "pairwise.complete.obs")),
           file.path(TABLES_DIR, "q7_twfe_correlations.xlsx"))
```

Time component of i removed by TWFE

```
year_mean_i   <- tapply(twfe_data$i, twfe_data$year, mean, na.rm = TRUE)
global_mean_i <- mean(twfe_data$i, na.rm = TRUE)
time_comp_i   <- data.frame(year = as.integer(names(year_mean_i)),
                            minus_year_mean_plus_global_mean_i = -year_mean_i + global_mean_i)
write_xlsx(time_comp_i, file.path(TABLES_DIR, "q7_twfe_time_component_i.xlsx"))
png(file.path(FIGURES_DIR, "q7_twfe_time_component_i.png"), width = 900, height = 500, res = 150)
plot(time_comp_i$year, time_comp_i$minus_year_mean_plus_global_mean_i,

     type = "o", pch = 16,
     main = "Two-way fixed effects time component for i",
     xlab = "Year", ylab = "minus i_.t plus i_..")
abline(h = 0, lty = 3); grid()

dev.off()
```

```
for (var in KEY_VARS_Q7) {
  save_distribution_q7(twfe_data[[paste0("twfe_", var)]],
    title = paste("Two-way fixed effects distribution of", var),
    filename = paste0("q7_twfe_distribution_", var, ".png"),
    xlabel = paste("Two-way fixed effects transformation of", var))
}

for (y_var in Y_VARS_Q7) {
  save_scatter_with_marginals_q7(twfe_data,
    x_col = paste0("twfe_", X_VAR_Q7), y_col = paste0("twfe_", y_var),
    title = paste("Two-way fixed effects:", X_VAR_Q7, "and", y_var),
    filename = paste0("q7_twfe_scatter_", X_VAR_Q7, "_", y_var, ".png"))
}
```

```
for (var in KEY_VARS_Q7) {
  save_country_boxplot_q7(twfe_data, paste0("twfe_", var),
    paste0("q7_twfe_boxplot_by_country_", var, ".png"), paste0("twfe_", var))
}

for (y_var in Y_VARS_Q7) {
  cc <- country_correlation_table_q7(twfe_data,
          paste0("twfe_", X_VAR_Q7), paste0("twfe_", y_var))
  write_xlsx(cc, file.path(TABLES_DIR,
    paste0("q7_twfe_country_correlations_", X_VAR_Q7, "_", y_var, ".xlsx")))
}
```

---- Unbalanced panel TWFE ----

```
unbalanced_data <- q7_data %>% filter(if_all(all_of(KEY_VARS_Q7), ~!is.na(.)))
obs_unbal       <- unbalanced_data %>% group_by(country) %>%
  summarise(T = n_distinct(year), .groups = "drop")
keep_countries    <- obs_unbal$country[obs_unbal$T > 1]
removed_countries <- sort(setdiff(unique(na.omit(q7_data$country)), keep_countries))
unbalanced_data <- unbalanced_data %>% filter(country %in% keep_countries) %>%
  arrange(country, year) %>% as.data.frame()
write_xlsx(data.frame(
  item  = c("Number of countries","Minimum number of years by country",
            "Maximum number of years by country","First year","Last year",
            "Number of observations","Countries removed because of one observation"),
  value = c(length(unique(unbalanced_data$country)),
            min(obs_unbal$T[obs_unbal$country %in% keep_countries]),
            max(obs_unbal$T[obs_unbal$country %in% keep_countries]),
            min(unbalanced_data$year), max(unbalanced_data$year),
            nrow(unbalanced_data),
            if (length(removed_countries)) paste(removed_countries, collapse = ", ") else "None")),
  file.path(TABLES_DIR, "q7_unbalanced_twfe_sample_summary.xlsx"))
for (var in KEY_VARS_Q7) {
  cmean <- ave(unbalanced_data[[var]], unbalanced_data$country, FUN = function(x) mean(x, na.rm = TRUE))
  wval  <- unbalanced_data[[var]] - cmean
  ymean <- ave(wval, unbalanced_data$year, FUN = function(x) mean(x, na.rm = TRUE))
  unbalanced_data[[paste0("within_unbalanced_",  var)]] <- wval
  unbalanced_data[[paste0("twfe_unbalanced_",    var)]] <- wval - ymean
}

unbal_twfe_vars <- paste0("twfe_unbalanced_", KEY_VARS_Q7)
write_xlsx(unbalanced_data[, c("country","year", KEY_VARS_Q7, unbal_twfe_vars)],
           file.path(TABLES_DIR, "q7_unbalanced_twfe_transformed_variables.xlsx"))
write_xlsx(as.data.frame(cor(unbalanced_data[unbal_twfe_vars], use = "pairwise.complete.obs")),
           file.path(TABLES_DIR, "q7_unbalanced_twfe_correlations.xlsx"))
for (var in KEY_VARS_Q7) {
  save_distribution_q7(unbalanced_data[[paste0("twfe_unbalanced_", var)]],
    title = paste("Unbalanced TWFE distribution of", var),
    filename = paste0("q7_unbalanced_twfe_distribution_", var, ".png"),
    xlabel = paste("Unbalanced TWFE transformation of", var))
}
```

cat("Question 7 outputs saved.\n")

==

10. QUESTION 8 — COMPARISON OF TRANSFORMED VARIABLES

==

```
Q8_VARS <- c("WR", "LS", "i")
Y_VARS  <- c("WR", "LS")
X_VAR   <- "i"
q8_between <- between_country
q8_within  <- bw_data[, c("country","year", paste0(Q8_VARS, "_within"))]
q8_fd      <- q7_data[,  c("country","year", paste0("d_",    Q8_VARS))]
q8_twfe    <- twfe_data[, c("country","year", paste0("twfe_", Q8_VARS))]
```

---- Q8.2 Summary statistics ----

R has no describe().T equivalent so we build the stats table manually.

```
q8_summary_rows <- list()
for (var in Q8_VARS) {
  transformations <- list(
    "Between"              = q8_between[[paste0(var, "_between")]],
    "One-way within"       = q8_within[[paste0(var,  "_within")]],
```

```
    "First differences"     = q8_fd[[paste0("d_",     var)]],
    "Two-way fixed effects" = q8_twfe[[paste0("twfe_", var)]]
  )
  for (trans_name in names(transformations)) {
    vals <- na.omit(transformations[[trans_name]])
    vals <- vals[is.finite(vals)]
    n    <- length(vals); s <- sd(vals)
    q8_summary_rows[[length(q8_summary_rows) + 1]] <- list(
      Variable = var, Transformation = trans_name, N = n,
      Mean = mean(vals), Median = median(vals),
      `Standard deviation` = s,
      `Standard error`     = if (n > 0) s / sqrt(n) else NA_real_,
      Q1 = quantile(vals, 0.25), Q3 = quantile(vals, 0.75),
      `Standardized min`   = if (s != 0) (min(vals) - mean(vals)) / s else NA_real_,
      `Standardized max`   = if (s != 0) (max(vals) - mean(vals)) / s else NA_real_
    )
  }
}

write_xlsx(bind_rows(q8_summary_rows),
           file.path(TABLES_DIR, "q8_transformation_summary_statistics.xlsx"))
# ---- Q8.3 Boxplots ----

save_q8_single_boxplot <- function(series, title, ylabel, filename) {
  vals <- na.omit(series); vals <- vals[is.finite(vals)]
  if (length(vals) < 3) { cat("Not enough data:", title, "\n"); return(invisible(NULL)) }
  png(file.path(FIGURES_DIR, filename), width = 600, height = 500, res = 150)
  boxplot(vals, main = title, ylab = ylabel, names = "All countries")
  abline(h = 0, lty = 3)
  dev.off()
}

save_q8_boxplot_by_country <- function(data, value_col, title, filename) {
  pd <- na.omit(data[, c("country", value_col)])
  pd <- pd[is.finite(pd[[value_col]]), ]
  if (nrow(pd) == 0) return(invisible(NULL))
  country_var <- tapply(pd[[value_col]], pd$country, var, na.rm = TRUE)
  ordered     <- names(sort(country_var))
  vals_list   <- lapply(ordered, function(ctry) pd[[value_col]][pd$country == ctry])
  png(file.path(FIGURES_DIR, filename), width = 1100, height = 600, res = 150)
  boxplot(vals_list, names = ordered, las = 2,
          main = title, xlab = "Country", ylab = value_col)
  abline(h = 0, lty = 3)
  dev.off()
}

for (var in Q8_VARS) {
  save_q8_single_boxplot(q8_between[[paste0(var, "_between")]],
    paste("Q8 Between distribution across countries:", var),
    paste0(var, "_between"), paste0("q8_boxplot_between_all_countries_", var, ".png"))
  save_q8_boxplot_by_country(q8_within, paste0(var, "_within"),
    paste("Q8 One-way within by country:", var),
    paste0("q8_boxplot_within_by_country_", var, ".png"))
  save_q8_boxplot_by_country(q8_fd, paste0("d_", var),
    paste("Q8 First differences by country:", var),
    paste0("q8_boxplot_fd_by_country_", var, ".png"))
  save_q8_boxplot_by_country(q8_twfe, paste0("twfe_", var),
    paste("Q8 Two-way fixed effects by country:", var),
    paste0("q8_boxplot_twfe_by_country_", var, ".png"))
}

# ---- Q8.4 Correlation matrices ----

# No seaborn in R. We use image() with a manual blue-white-red colour ramp

# and add the cell annotations with text(). More code, same result.

save_corr_heatmap <- function(corr_matrix, title, filename) {
  n      <- nrow(corr_matrix)
  n_col  <- 101
  breaks <- seq(-1, 1, length.out = n_col + 1)
  cols   <- colorRampPalette(c("#3B4CC0", "white", "#B40426"))(n_col)
  png(file.path(FIGURES_DIR, filename), width = 700, height = 650, res = 150)
  par(mar = c(8, 8, 4, 5))
  image(1:n, 1:n, t(corr_matrix[n:1, ]),
        col = cols, breaks = breaks, axes = FALSE,
        xlab = "", ylab = "", main = title)
  axis(1, at = 1:n, labels = colnames(corr_matrix), las = 2, cex.axis = 0.9)
```

```
  axis(2, at = 1:n, labels = rev(rownames(corr_matrix)), las = 1, cex.axis = 0.9)
  for (i in seq_len(n)) for (j in seq_len(n)) {
    text(j, n + 1 - i, round(corr_matrix[i, j], 2), cex = 0.8)
  }
  dev.off()
}

# Correlation matrices for WR, LS, i only
q8_between_corr <- cor(q8_between[, paste0(Q8_VARS, "_between")], use = "complete.obs")
q8_within_corr  <- cor(q8_within[,  paste0(Q8_VARS, "_within")],  use = "complete.obs")
q8_fd_corr      <- cor(q8_fd[,      paste0("d_",    Q8_VARS)],    use = "complete.obs")
q8_twfe_corr    <- cor(q8_twfe[,    paste0("twfe_", Q8_VARS)],    use = "complete.obs")
for (pair in list(
  list(q8_between_corr, "Between Correlation Matrix",              "q8_between_correlation_matrix"),
  list(q8_within_corr,  "One-Way Within Correlation Matrix",       "q8_within_correlation_matrix"),
  list(q8_fd_corr,      "First Difference Correlation Matrix",     "q8_fd_correlation_matrix"),
  list(q8_twfe_corr,    "Two-Way Fixed Effects Correlation Matrix", "q8_twfe_correlation_matrix")
)) {

  write_xlsx(as.data.frame(pair[[1]]), file.path(TABLES_DIR, paste0(pair[[3]], ".xlsx")))
  save_corr_heatmap(pair[[1]], pair[[2]], paste0(pair[[3]], ".png"))
}
```

Full correlation matrices with trend and lags

```
Q8_CORR_VARS <- setdiff(ALL_VARIABLES, "PCOM")
Q8_CORR_VARS <- Q8_CORR_VARS[Q8_CORR_VARS %in% names(df)]
q8_corr_base <- df %>% select(country, year, all_of(Q8_CORR_VARS)) %>%
  group_by(country) %>%
  mutate(trend = row_number(),
         across(all_of(Q8_CORR_VARS), list(lag1 = ~lag(.)), .names = "{.col}_lag1")) %>%
  ungroup() %>% as.data.frame()
between_corr_cols <- c(Q8_CORR_VARS, "trend", paste0(Q8_CORR_VARS, "_lag1"))
between_corr_cols <- intersect(between_corr_cols, names(q8_corr_base))
between_corr_data <- q8_corr_base %>% group_by(country) %>%
  summarise(across(where(is.numeric), ~mean(.x, na.rm = TRUE)), .groups = "drop") %>%
  as.data.frame()
q8_between_full_corr <- cor(between_corr_data[, intersect(between_corr_cols, names(between_corr_data))],
                            use = "pairwise.complete.obs")
write_xlsx(as.data.frame(q8_between_full_corr),
           file.path(TABLES_DIR, "q8_between_full_correlation_matrix_with_trend_lags.xlsx"))
within_corr_data <- q8_corr_base
for (col in intersect(between_corr_cols, names(q8_corr_base))) {
  within_corr_data[[paste0(col, "_within")]] <-
    within_corr_data[[col]] - ave(within_corr_data[[col]], within_corr_data$country,
                                  FUN = function(x) mean(x, na.rm = TRUE))
}

within_corr_cols <- paste0(intersect(between_corr_cols, names(q8_corr_base)), "_within")
within_corr_cols <- intersect(within_corr_cols, names(within_corr_data))
q8_within_full_corr <- cor(within_corr_data[, within_corr_cols], use = "pairwise.complete.obs")
write_xlsx(as.data.frame(q8_within_full_corr),
           file.path(TABLES_DIR, "q8_within_full_correlation_matrix_with_trend_lags.xlsx"))
fd_corr_data <- q7_data[, c("country", "year")]
for (var in Q8_CORR_VARS) {
  d_col <- paste0("d_", var)
  fd_corr_data[[d_col]] <- if (d_col %in% names(q7_data)) q7_data[[d_col]] else
    ave(q7_data[[var]], q7_data$country,
        FUN = function(x) c(NA, diff(x)))
  fd_corr_data[[paste0(d_col, "_lag1")]] <-
    ave(fd_corr_data[[d_col]], fd_corr_data$country,
        FUN = function(x) c(NA, head(x, -1)))
}

fd_corr_cols <- intersect(c(paste0("d_", Q8_CORR_VARS), paste0("d_", Q8_CORR_VARS, "_lag1")),
                          names(fd_corr_data))
q8_fd_full_corr <- cor(fd_corr_data[, fd_corr_cols], use = "pairwise.complete.obs")
write_xlsx(as.data.frame(q8_fd_full_corr),
           file.path(TABLES_DIR, "q8_fd_full_correlation_matrix_with_lags.xlsx"))
twfe_corr_data <- twfe_data[, c("country", "year")]
for (var in Q8_CORR_VARS) {
  tc <- paste0("twfe_", var)
  twfe_corr_data[[tc]] <- if (tc %in% names(twfe_data)) twfe_data[[tc]] else
    if (var %in% names(twfe_data)) twfe_transform(twfe_data, var) else NA_real_
}

twfe_corr_cols <- grep("^twfe_", names(twfe_corr_data), value = TRUE)
q8_twfe_full_corr <- cor(twfe_corr_data[, twfe_corr_cols], use = "pairwise.complete.obs")
```

```
write_xlsx(as.data.frame(q8_twfe_full_corr),
           file.path(TABLES_DIR, "q8_twfe_full_correlation_matrix.xlsx"))
# ---- Q8.5 Autocorrelation and trend correlation ----

auto_trend_rows <- lapply(Q8_CORR_VARS, function(var) {
  lag1_col   <- paste0(var, "_lag1")
  temp_auto  <- na.omit(q8_corr_base[, c(var, lag1_col)])
  temp_trend <- na.omit(q8_corr_base[, c(var, "trend")])
  list(Variable = var,
       `Autocorrelation with lag 1` = cor(temp_auto[[var]], temp_auto[[lag1_col]]),
       `N autocorrelation`          = nrow(temp_auto),
       `Trend correlation`          = cor(temp_trend[[var]], temp_trend$trend),
       `N trend correlation`        = nrow(temp_trend))
})

write_xlsx(bind_rows(auto_trend_rows),
           file.path(TABLES_DIR, "q8_autocorrelation_and_trend_correlation.xlsx"))
# First 30 FD observations with lags

fd_lag_cols <- intersect(c("country","year","d_WR","d_LS","d_i","d_WR_lag1","d_LS_lag1","d_i_lag1"),
                         names(fd_corr_data))
write_xlsx(head(fd_corr_data[, fd_lag_cols], 30),
           file.path(TABLES_DIR, "q8_first_30_fd_and_lag_check.xlsx"))
# ---- Q8.6 Bivariate graphs: linear + quadratic + LOWESS ----

# lowess() is in base R so no conditional import is needed unlike Python.
save_q8_bivariate_fit <- function(data, x_col, y_col, title, filename) {
  pd <- na.omit(data[, c(x_col, y_col)])
  pd <- pd[is.finite(pd[[x_col]]) & is.finite(pd[[y_col]]), ]
  if (nrow(pd) < 5 || length(unique(pd[[x_col]])) <= 2) {
    cat("Not enough data for graph:", title, "\n")
    return(invisible(NULL))
  }
  x <- pd[[x_col]]; y <- pd[[y_col]]
  corr   <- cor(x, y)
  x_grid <- seq(min(x), max(x), length.out = 200)
  png(file.path(FIGURES_DIR, filename), width = 800, height = 500, res = 150)
  plot(x, y, pch = 16, col = rgb(0, 0, 0, 0.55),
       main = paste0(title, "\nCorrelation = ", round(corr, 3)),
       xlab = x_col, ylab = y_col)
  lines(x_grid, coef(lm(y ~ x))[1] + coef(lm(y ~ x))[2] * x_grid,
        col = "blue", lwd = 2)
  if (length(unique(x)) > 2) {
    qc <- coef(lm(y ~ x + I(x^2)))
    lines(x_grid, qc[1] + qc[2] * x_grid + qc[3] * x_grid^2,
          col = "orange", lwd = 2)
  }
  lw <- lowess(x, y, f = 0.3)   # base R lowess — no extra package needed
  lines(lw$x, lw$y, col = "darkgreen", lwd = 2)
  legend("topright",
         legend = c("Linear fit", "Quadratic fit", "LOWESS fit"),
         col = c("blue", "orange", "darkgreen"), lty = 1, lwd = 2)
  grid()
  dev.off()
}

for (y_var in Y_VARS) {
  save_q8_bivariate_fit(q8_between, paste0(X_VAR,"_between"), paste0(y_var,"_between"),
    paste("Q8 Between:", X_VAR, "and", y_var),
    paste0("q8_bivariate_between_", X_VAR, "_", y_var, ".png"))
  save_q8_bivariate_fit(q8_within, paste0(X_VAR,"_within"), paste0(y_var,"_within"),
    paste("Q8 One-way within:", X_VAR, "and", y_var),
    paste0("q8_bivariate_within_", X_VAR, "_", y_var, ".png"))
  save_q8_bivariate_fit(q8_fd, paste0("d_",X_VAR), paste0("d_",y_var),
    paste0("Q8 First differences: d_", X_VAR, " and d_", y_var),
    paste0("q8_bivariate_fd_", X_VAR, "_", y_var, ".png"))
  save_q8_bivariate_fit(q8_twfe, paste0("twfe_",X_VAR), paste0("twfe_",y_var),
    paste("Q8 TWFE:", X_VAR, "and", y_var),
    paste0("q8_bivariate_twfe_", X_VAR, "_", y_var, ".png"))
}

cat("Question 8 outputs saved.\n")

# ============================================================

# 11. QUESTION 9 — COUNTRY HETEROGENEITY
```

```
# ============================================================

classify_correlation <- function(corr) {
  if (is.na(corr)) return("Missing")
  if (corr > 0.08)  return("Positive")
  if (corr < -0.08) return("Negative")
  return("Weak")
}

q9_rows <- list()
for (y_var in Y_VARS) {
  for (trans in c("fd", "twfe")) {
    dataset  <- if (trans == "fd")   q7_data   else twfe_data
    y_col    <- if (trans == "fd")   paste0("d_",    y_var) else paste0("twfe_", y_var)
    x_col    <- if (trans == "fd")   "d_i"                  else "twfe_i"
    trans_lbl<- if (trans == "fd")   "First differences"    else "Two-way fixed effects"
    for (ctry in unique(na.omit(dataset$country))) {
      s     <- dataset[dataset$country == ctry, c(y_col, x_col)]
      s     <- na.omit(s); s <- s[is.finite(s[[1]]) & is.finite(s[[2]]), ]
      n_obs <- nrow(s)
      if (n_obs >= 3 && sd(s[[2]], na.rm = TRUE) != 0) {
        corr  <- cor(s[[1]], s[[2]]); std_y <- sd(s[[1]]); std_x <- sd(s[[2]])
        beta  <- corr * std_y / std_x
      } else {
        corr <- std_y <- std_x <- beta <- NA_real_
      }
      q9_rows[[length(q9_rows) + 1]] <- list(
        `Dependent variable`             = y_var,
        Transformation                   = trans_lbl,
        `Individual name (i)`            = ctry,
        `T(i)`                           = n_obs,
        `r(Y,X)`                         = corr,
        `sigma(Y)`                       = std_y,
        `sigma(X)`                       = std_x,
        `sigma(Y)/sigma(X)`              = if (!is.na(std_x) && std_x != 0) std_y / std_x else NA_real_,
        `beta = r * sigma(Y)/sigma(X)`   = beta,
        Group                            = classify_correlation(corr)
      )
    }
  }
}

q9_country_heterogeneity <- bind_rows(q9_rows) %>%
  arrange(`Dependent variable`, Transformation, desc(`r(Y,X)`))
write_xlsx(q9_country_heterogeneity,
           file.path(TABLES_DIR, "q9_country_heterogeneity_fd_twfe.xlsx"))
q9_group_diagnosis <- q9_country_heterogeneity %>%
  group_by(`Dependent variable`, Transformation, Group) %>%
  summarise(`Number of countries` = n(), .groups = "drop")
write_xlsx(q9_group_diagnosis, file.path(TABLES_DIR, "q9_group_diagnosis_summary.xlsx"))
cat("Question 9 outputs saved.\n")

# ============================================================

# 12. QUESTION 10 — PANEL ESTIMATORS

# ============================================================

# Python's linearmodels takes separate y and X objects. plm uses the

# standard R formula interface which is more concise. All five estimators

# map directly: Between, FE, TWFE, FD, and Mundlak.

df_panel <- pdata.frame(
  read_excel(CLEAN_DATA_FILE),
  index = c("country", "year")
)

dep_vars  <- c("LS", "WR")
expl_vars <- c("i", "GDP", "UN", "REER")
formula_base <- as.formula(paste("~", paste(expl_vars, collapse = " + ")))
for (dep_var in dep_vars) {
  cat("\n==================================================\n")
  cat("RESULTS FOR DEPENDENT VARIABLE:", dep_var, "\n")
  cat("==================================================\n")
```

```
  full_formula <- update(formula_base, paste(dep_var, "~ ."))
  # 1. Between estimator
  # plm model="between" averages within groups before estimating.
  # Equivalent to linearmodels BetweenOLS.
  between_res <- summary(plm(full_formula, data = df_panel, model = "between"))
  cat("\n--- BETWEEN ESTIMATOR ---\n"); print(between_res)
  capture.output(print(between_res),
    file = file.path(TABLES_DIR, paste0("q10_between_results_", dep_var, ".txt")))
  # 2. Fixed effects (one-way within)
  # entity_effects only, equivalent to PanelOLS(entity_effects=True)
  fe_res <- summary(plm(full_formula, data = df_panel,
                        model = "within", effect = "individual"))
  cat("\n--- FIXED EFFECTS (WITHIN) ---\n"); print(fe_res)
  capture.output(print(fe_res),
    file = file.path(TABLES_DIR, paste0("q10_fe_results_", dep_var, ".txt")))
  # 3. Two-way fixed effects
  # effect="twoways" adds both entity and time fixed effects.
  # Equivalent to PanelOLS(entity_effects=True, time_effects=True).
  twfe_res <- summary(plm(full_formula, data = df_panel,
                          model = "within", effect = "twoways"))
  cat("\n--- TWO-WAY FIXED EFFECTS ---\n"); print(twfe_res)
  capture.output(print(twfe_res),
    file = file.path(TABLES_DIR, paste0("q10_twfe_results_", dep_var, ".txt")))
  # 4. First differences
  # plm model="fd" handles the differencing automatically, which is
  # cleaner than the manual diff approach used in the Python version.
  fd_res <- summary(plm(full_formula, data = df_panel, model = "fd"))
  cat("\n--- FIRST DIFFERENCES ---\n"); print(fd_res)
  capture.output(print(fd_res),
    file = file.path(TABLES_DIR, paste0("q10_fd_results_", dep_var, ".txt")))
  # 5. Mundlak / Correlated Random Effects
  # Same idea as Python: compute group means and add them as extra regressors,
  # then run random effects. plm::Between() extracts the group mean from a
  # pdata.frame, which is the equivalent of groupby().transform("mean").
  df_mundlak <- df_panel
  for (var in expl_vars) {
    df_mundlak[[paste0(var, "_mean")]] <- Between(df_panel[[var]], effect = "individual")
  }
  mundlak_formula <- as.formula(
    paste(dep_var, "~",
          paste(c(expl_vars, paste0(expl_vars, "_mean")), collapse = " + "))
  )
  mundlak_res <- summary(plm(mundlak_formula, data = df_mundlak, model = "random"))
  cat("\n--- MUNDLAK RANDOM EFFECTS ---\n"); print(mundlak_res)
  capture.output(print(mundlak_res),
    file = file.path(TABLES_DIR, paste0("q10_mundlak_results_", dep_var, ".txt")))
}

cat("\nAll estimations completed successfully.\n")

# ============================================================

# Path detection: Python finds the script automatically via __file__.

# The formulas match pandas exactly so the numbers are the same.

#

# Heatmaps: seaborn does this in one line. In R we had to build the

# colour ramp manually with image() and add annotations with text().

#

# Scatter plots with marginal histograms: matplotlib's GridSpec makes

# this straightforward.

#

# KDE: density() in R is the equivalent of scipy's gaussian_kde.

# The default bandwidths differ slightly but visually it is the same.
```

```
#
# Grouped differences: the data needs to be sorted by year before
# using lag() inside group_by(), otherwise the differences are wrong.
# Python's groupby().diff() handles this internally.
#
# Panel models: linearmodels takes y and X as separate objects.
# plm uses R's formula interface which is more concise. All five
# estimators translate directly with no loss of functionality.
# LOWESS: always in base R so no conditional import is needed, unlike
# the Python version which catches an ImportError at runtime.
# ============================================================
```

STATA code:

```
* ============================================================
* Panel Data Replication Project
* Lofaro and Di Bucchianico (2025)
* Stata translation (with LLM assistance)
*
* KNOWN DIFFICULTIES / DIFFERENCES FROM PYTHON:
* 1. Stata works with one dataset in memory at a time. The Python
*    script builds several dataframes simultaneously (between_country,
*    q7_data, twfe_data, etc.). In Stata we must save, clear, reload,
*    and merge repeatedly. Every section below notes when this happens.
* 2. Stata has no native KDE histogram overlay as clean as matplotlib.
*    We use -kdensity- and -twoway- to approximate it.
* 3. Scatter plots with marginal histograms (save_scatter_with_marginals_q7)
*    cannot be replicated natively; we produce plain scatterplots instead.
* 4. The LOWESS bivariate graphs use -lowess- which is available in
*    Stata but the three-fit overlay (linear + quadratic + LOWESS) must
*    be built with -twoway- combining multiple plot types.
* 5. Correlation heatmaps do not exist natively in Stata. We export the
*    correlation matrices to Excel and note that visualisation requires
*    an external tool or the user-written command -heatplot- (SSC).
* 6. The Mundlak/CRE estimator uses -xtreg, re- with added group means,
```

```
*    not a dedicated command. This is equivalent to the Python approach.

* 7. Path handling: Stata uses macros instead of Path objects. Adjust

*    the BASE_DIR macro below to match your folder layout before running.

* 8. Excel export uses -export excel-, which requires Stata 12+.

* 9. -xtset- requires numeric IDs; we encode the string country variable.

* ============================================================

* ============================================================

* 0. USER SETTINGS — adjust before running

* ============================================================

* Set your project root. All other paths are built from this.
* Example: global BASE_DIR "C:/Users/ann/Documents/panel_project"
global BASE_DIR "REPLACE_WITH_YOUR_PROJECT_ROOT"
global DATA_DIR    "$BASE_DIR/02_original_data"
global CLEAN_DIR   "$BASE_DIR/03_clean_data"
global TABLES_DIR  "$BASE_DIR/08_tables"
global FIGURES_DIR "$BASE_DIR/07_figures"
* Create output folders (shell commands — works on Windows and Mac/Linux)

cap mkdir "$CLEAN_DIR"

cap mkdir "$TABLES_DIR"
cap mkdir "$FIGURES_DIR"
global DATA_FILE       "$DATA_DIR/Dataset_MP_Impact_functional_Distribution.xlsx"
global CLEAN_DATA_FILE "$CLEAN_DIR/Dataset_MP_Impact_functional_Distribution_clean.dta"
* Variable lists

global DEPENDENT_VARS  WR LS
global KEY_X           i
global ALL_VARIABLES   i P W WR GDP LS PCOM UN SHORTUN LONGUN LF REER SH
global EXPL_VARS       i GDP UN REER
* ============================================================

* 1. LOAD AND CLEAN DATA

* ============================================================

import excel using "$DATA_FILE", sheet("Sheet1") firstrow clear
* Trim variable names (Stata does not store spaces in varnames,

* but labels might have leading/trailing spaces)

* Sort panel

sort country year

* Save clean version

save "$CLEAN_DATA_FILE", replace

* Keep only variables that exist (drop those not in the sheet)

* DIFFICULTY: Python does this dynamically. In Stata we check manually

* or use a loop with -cap confirm variable-.

foreach var of global ALL_VARIABLES {
    cap confirm variable `var'
    if _rc != 0 {
        di "WARNING: variable `var' not found in dataset — skipping"
```

```
    }
}

* Encode string country to numeric ID required by -xtset-

encode country, gen(country_id)

xtset country_id year

di "Dataset loaded."

di "Observations: `=_N'"

* ============================================================

* HELPER: consecutive_blocks equivalent

* We use this logic inline where needed (max run of non-missing obs).

* DIFFICULTY: Python's consecutive_blocks() function has no direct Stata

* equivalent. We approximate it by checking year gaps within country.

* ============================================================

* ============================================================

* QUESTIONS 1–6: SAMPLE SELECTION

* ============================================================

* --- Max consecutive non-missing observations per country × dep var ---

* DIFFICULTY: This requires a within-group run-length calculation.

* We implement it with a loop generating a run counter.

tempfile main_data

save `main_data'

foreach dep in $DEPENDENT_VARS {
    use `main_data', clear
    keep country country_id year `dep'
    sort country_id year
    * Mark non-missing years
    gen nonmiss = !missing(`dep')
    * Run-length counter within country (reset on missing or new country)
    by country_id: gen run = 1 if nonmiss == 1 & (_n == 1 | nonmiss[_n-1] == 0)
    by country_id: replace run = run[_n-1] + 1 if nonmiss == 1 & _n > 1 & nonmiss[_n-1] == 1
    replace run = 0 if missing(run)
    * Max consecutive per country
    by country_id: egen max_consec_`dep' = max(run)
    * Total non-missing
    by country_id: egen n_nonmiss_`dep' = total(nonmiss)
    * First and last available year
    by country_id: egen first_yr_`dep' = min(cond(!missing(`dep'), year, .))
    by country_id: egen last_yr_`dep'  = max(cond(!missing(`dep'), year, .))
    gen kept_`dep' = (max_consec_`dep' >= 3)
    * Keep one row per country
    by country_id: keep if _n == 1
    keep country country_id max_consec_`dep' n_nonmiss_`dep' ///
         first_yr_`dep' last_yr_`dep' kept_`dep'
    tempfile sel_`dep'
    save `sel_`dep''
}

* Merge selection tables
```

```
use `sel_WR', clear

merge 1:1 country_id using `sel_LS', nogen

export excel using "$TABLES_DIR/sample_selection.xlsx", ///
    sheet("sample_selection") firstrow(variables) replace
* Summary counts

count

local total_countries = r(N)

count if kept_WR == 0 | kept_LS == 0
local n_excluded = r(N)

count if kept_WR == 1 & kept_LS == 1
local n_kept = r(N)

di "Total countries: `total_countries'"

di "Excluded: `n_excluded'"

di "Kept: `n_kept'"

* ============================================================

* Number of countries per year (WR)

* ============================================================

use `main_data', clear

gen nonmiss_WR = !missing(WR)
preserve

    keep if nonmiss_WR == 1
    collapse (count) n_countries=country_id, by(year)
    export excel using "$TABLES_DIR/number_of_countries_per_year.xlsx", ///
        sheet("countries_per_year") firstrow(variables) replace
    twoway connected n_countries year, ///
        title("Number of countries observed per year") ///
        xtitle("Year") ytitle("Number of countries") ///
        msymbol(circle)
    graph export "$FIGURES_DIR/number_of_countries_per_year.png", replace width(1000)
restore

* ============================================================

* Observations by country and holes

* ============================================================

preserve

    keep if !missing(WR)
    bysort country_id: gen n_obs = _N
    bysort country_id: gen first_yr = year[1]
    bysort country_id: gen last_yr  = year[_N]
    * Detect holes: expected span minus actual obs
    gen span = last_yr - first_yr + 1
    gen n_missing_inside = span - n_obs
    gen has_holes = (n_missing_inside > 0)
    by country_id: keep if _n == 1
    keep country n_obs first_yr last_yr has_holes n_missing_inside
    export excel using "$TABLES_DIR/holes_inside_panel.xlsx", ///
        sheet("holes") firstrow(variables) replace
    * Holes summary
    sum has_holes
    local n_holes    = r(sum)
    local prop_holes = r(mean)
    di "Countries with holes: `n_holes', proportion: `prop_holes'"
```

```
restore

* ============================================================

* WITHIN / BETWEEN VARIANCE DECOMPOSITION

* ============================================================

* DIFFICULTY: Stata's -xtsum- gives within/between/overall std dev

* automatically. We use it for all variables and reconstruct variances.
use `main_data', clear

* Build variance table for each variable
tempfile var_table

cap erase `var_table'

foreach var in $ALL_VARIABLES {
    cap confirm variable `var'
    if _rc != 0 continue
    qui xtsum `var'
    local overall_var = r(sd_overall)^2
    local between_var = r(sd_b)^2
    local within_var  = r(sd_w)^2
    local within_share = `within_var' / `overall_var'
    * Append one row
    clear
    set obs 1
    gen variable      = "`var'"
    gen overall_var   = `overall_var'
    gen between_var   = `between_var'
    gen within_var    = `within_var'
    gen within_share  = `within_share'
    gen within_pct    = `within_share' * 100
    cap append using `var_table'
    save `var_table', replace
}

use `var_table', clear

sort within_share

export excel using "$TABLES_DIR/within_between_variance_all_variables.xlsx", ///
    sheet("variance") firstrow(variables) replace
* ============================================================

* Variable categories (time-invariant, individual-invariant, two-index)

* ============================================================

* DIFFICULTY: In Python this is a loop over a DataFrame.

* In Stata we use -xtsum- output directly.

use `main_data', clear

foreach var in $ALL_VARIABLES {
    cap confirm variable `var'
    if _rc != 0 continue
    qui xtsum `var'
    local ws = (r(sd_w)^2) / (r(sd_overall)^2)
    if abs(`ws') < 1e-6 {
        di "`var' -> TIME INVARIANT"
    }
    else if abs(`ws' - 1) < 1e-6 {
        di "`var' -> INDIVIDUAL INVARIANT (common time series)"
    }
    else {
        di "`var' -> TWO-INDEX (varies across countries and time)"
    }
}
```

```
* ============================================================

* BETWEEN AND ONE-WAY WITHIN DECOMPOSITION

* ============================================================

use `main_data', clear

foreach var in WR LS i {
    by country_id: egen `var'_between = mean(`var')
    gen `var'_within = `var' - `var'_between
}

save "$TABLES_DIR/between_within_variables.dta", replace
* Distribution plots (histogram + kdensity + normal overlay)

* DIFFICULTY: matplotlib produces these in one call. In Stata we use

* -twoway (histogram) (kdensity) (function)- for the overlay.
foreach var in WR LS i {
    * Between distribution
    preserve
        by country_id: keep if _n == 1
        keep country `var'_between
        drop if missing(`var'_between)
        qui sum `var'_between
        local mu  = r(mean)
        local sig = r(sd)
        twoway ///
            (histogram `var'_between, freq density color(blue%40) lcolor(blue%40)) ///
            (kdensity  `var'_between, lcolor(blue) lwidth(medthick)) ///
            (function  y = normalden(x, `mu', `sig'), ///
                range(`var'_between) lcolor(red) lwidth(medthick)), ///
            title("Between distribution of `var'") ///
            xtitle("`var' country average") ytitle("Density") ///
            legend(order(1 "Histogram" 2 "KDE" 3 "Normal"))
        graph export "$FIGURES_DIR/between_distribution_`var'.png", replace width(800)
    restore
    * Within distribution
    preserve
        keep if !missing(`var'_within)
        qui sum `var'_within
        local mu  = r(mean)
        local sig = r(sd)
        twoway ///
            (histogram `var'_within, freq density color(blue%40) lcolor(blue%40)) ///
            (kdensity  `var'_within, lcolor(blue) lwidth(medthick)) ///
            (function  y = normalden(x, `mu', `sig'), ///
                range(`var'_within) lcolor(red) lwidth(medthick)), ///
            title("One-way within distribution of `var'") ///
            xtitle("`var' minus country average") ytitle("Density") ///
            legend(order(1 "Histogram" 2 "KDE" 3 "Normal"))
        graph export "$FIGURES_DIR/within_distribution_`var'.png", replace width(800)
    restore
}

* Descriptive statistics for between and within
preserve

    * Between: one obs per country
    by country_id: keep if _n == 1
    foreach var in WR LS i {
        qui sum `var'_between, detail
        di "Between `var': mean=`r(mean)', sd=`r(sd)', min=`r(min)', max=`r(max)'"
    }
restore

foreach var in WR LS i {
    qui sum `var'_within, detail
    di "Within `var': mean=`r(mean)', sd=`r(sd)', min=`r(min)', max=`r(max)'"
}

* Bivariate scatterplots (between and within vs i)

preserve
```

```
    by country_id: keep if _n == 1
    foreach y in WR LS {
        qui corr `y'_between i_between
        local rho = r(rho)
        twoway (scatter `y'_between i_between) ///
               (lfit   `y'_between i_between), ///
            title("Between relationship between i and `y'" ///
                  "Correlation = `: display %5.3f `rho''") ///
            xtitle("i_between") ytitle("`y'_between") ///
            legend(order(2 "Linear fit"))
        graph export "$FIGURES_DIR/between_scatter_i_`y'.png", replace width(800)
    }
restore

foreach y in WR LS {
    qui corr `y'_within i_within
    local rho = r(rho)
    twoway (scatter `y'_within i_within) ///
           (lfit   `y'_within i_within), ///
        title("One-way within relationship between i and `y'" ///
              "Correlation = `: display %5.3f `rho''") ///
        xtitle("i_within") ytitle("`y'_within") ///
        legend(order(2 "Linear fit"))
    graph export "$FIGURES_DIR/within_scatter_i_`y'.png", replace width(800)
}

* ============================================================

* QUESTION 7: FIRST DIFFERENCES AND TWO-WAY FIXED EFFECTS

* ============================================================

use `main_data', clear

xtset country_id year

* ---- First differences ----

* DIFFICULTY: Python checks year_gap == 1 to avoid cross-country differences.

* Stata's -D.- operator on a panel already respects xtset gaps,

* returning missing for non-consecutive years — same behaviour.
foreach var in WR LS i {
    gen d_`var' = D.`var'
}

* Boundary check: show rows around country transitions

sort country_id year

list country year WR d_WR LS d_LS i d_i in 1/15, sepby(country_id)

export excel country year WR d_WR LS d_LS i d_i using ///

    "$TABLES_DIR/q7_first_difference_boundary_check.xlsx", ///
    sheet("boundary") firstrow(variables) replace
* FD descriptive statistics

foreach var in WR LS i {
    qui sum d_`var', detail
    di "d_`var': N=`r(N)', mean=`r(mean)', sd=`r(sd)', min=`r(min)', max=`r(max)'"
}

* FD distribution plots

foreach var in WR LS i {
    qui sum d_`var', detail
    local mu  = r(mean)
    local sig = r(sd)
    twoway ///
        (histogram d_`var', density color(blue%40) bins(30)) ///
```

```
        (kdensity  d_`var') ///
        (function  y = normalden(x, `mu', `sig'), range(d_`var')), ///
        title("First difference distribution of `var'") ///
        xtitle("First difference of `var'") ytitle("Density") ///
        legend(order(1 "Histogram" 2 "KDE" 3 "Normal"))
    graph export "$FIGURES_DIR/q7_fd_distribution_`var'.png", replace width(800)
}

* FD scatterplots

* DIFFICULTY: Scatter with marginal histograms not available natively.

* We produce plain scatterplots with a linear fit.

foreach y in WR LS {
    qui corr d_`y' d_i
    local rho = r(rho)
    twoway (scatter d_`y' d_i) (lfit d_`y' d_i), ///
        title("First differences: d_i and d_`y'" ///
              "Correlation = `: display %5.3f `rho''") ///
        xtitle("d_i") ytitle("d_`y'") legend(order(2 "Linear fit"))
    graph export "$FIGURES_DIR/q7_fd_scatter_d_i_d_`y'.png", replace width(800)
}

tempfile q7_data
save `q7_data'
* ---- Balanced panel TWFE ----

* DIFFICULTY: Python finds the balanced subsample algorithmically.

* In Stata we use -xtbalance- (SSC) or do it manually with -fillin-.

* We implement it manually below.

use `q7_data', clear
drop if missing(WR) | missing(LS) | missing(i)
* Count T per country

by country_id: gen T_i = _N
qui sum T_i
local max_t = r(max)

* Keep only countries with the maximum number of observations

keep if T_i == `max_t'
* Keep only years present for ALL remaining countries
by year: gen n_ctry_yr = _N
qui sum n_ctry_yr
local n_balanced = r(max)   // after filtering, should equal n countries

keep if n_ctry_yr == `n_balanced'
sort country_id year

* TWFE transformation: x_it - x_i. - x_.t + x_..

foreach var in WR LS i {
    by country_id: egen cmean_`var' = mean(`var')
    by year:        egen ymean_`var' = mean(`var')
    qui sum `var'
    local gmean_`var' = r(mean)
    gen twfe_`var' = `var' - cmean_`var' - ymean_`var' + `gmean_`var''
}

* TWFE summary

foreach var in WR LS i {
    qui sum twfe_`var', detail
    di "twfe_`var': mean=`r(mean)', sd=`r(sd)'"
}

* Time component of i
```

preserve

```
    by year: keep if _n == 1
    gen time_comp_i = -ymean_i + `gmean_i'
    twoway connected time_comp_i year, ///
        yline(0) title("Two-way fixed effects time component for i") ///
        xtitle("Year") ytitle("minus i_.t plus i_..")
    graph export "$FIGURES_DIR/q7_twfe_time_component_i.png", replace width(900)
```

restore

* TWFE distributions

```
foreach var in WR LS i {
    qui sum twfe_`var', detail
    local mu  = r(mean)
    local sig = r(sd)
    twoway ///
        (histogram twfe_`var', density color(blue%40) bins(30)) ///
        (kdensity  twfe_`var') ///
        (function  y = normalden(x, `mu', `sig'), range(twfe_`var')), ///
        title("TWFE distribution of `var'") ///
        xtitle("TWFE transformation of `var'") ytitle("Density") ///
        legend(order(1 "Histogram" 2 "KDE" 3 "Normal"))
    graph export "$FIGURES_DIR/q7_twfe_distribution_`var'.png", replace width(800)
}
```

* TWFE scatterplots

```
foreach y in WR LS {
    qui corr twfe_`y' twfe_i
    local rho = r(rho)
    twoway (scatter twfe_`y' twfe_i) (lfit twfe_`y' twfe_i), ///
        title("TWFE: i and `y'" "Correlation = `: display %5.3f `rho''") ///
        xtitle("twfe_i") ytitle("twfe_`y'") legend(order(2 "Linear fit"))
    graph export "$FIGURES_DIR/q7_twfe_scatter_i_`y'.png", replace width(800)
}
```

* Country boxplots (TWFE)

* DIFFICULTY: Python orders by within-country variance. In Stata we

* produce a single -graph box- command; ordering by variance requires

* pre-sorting and is more involved.

```
foreach var in WR LS i {
    graph box twfe_`var', over(country, sort(1)) ///
        title("TWFE boxplot of `var' by country") yline(0)
    graph export "$FIGURES_DIR/q7_twfe_boxplot_by_country_`var'.png", replace width(1100)
}
```

* Country-level correlations (TWFE)

```
foreach y in WR LS {
    tempfile corr_`y'
    cap erase `corr_`y''
    levelsof country, local(ctry_list)
    foreach ctry of local ctry_list {
        qui corr twfe_`y' twfe_i if country == "`ctry'"
        local rho_ctry = r(rho)
        clear
        set obs 1
        gen country_name = "`ctry'"
        gen correlation  = `rho_ctry'
        cap append using `corr_`y''
        save `corr_`y'', replace
    }
    use `corr_`y'', clear
    sort correlation
    export excel using "$TABLES_DIR/q7_twfe_country_correlations_i_`y'.xlsx", ///
        sheet("corr") firstrow(variables) replace
}
```

tempfile twfe_data

```
save `twfe_data'

* ---- Unbalanced panel TWFE ----

use `q7_data', clear
drop if missing(WR) | missing(LS) | missing(i)
* Remove countries with only one observation

by country_id: gen T_unbal = _N
drop if T_unbal <= 1
foreach var in WR LS i {
    by country_id: egen cmean_unbal_`var' = mean(`var')
    gen within_unbal_`var' = `var' - cmean_unbal_`var'
    by year: egen ymean_unbal_`var' = mean(within_unbal_`var')
    gen twfe_unbal_`var' = within_unbal_`var' - ymean_unbal_`var'
}

* Save

export excel country year WR LS i twfe_unbal_WR twfe_unbal_LS twfe_unbal_i ///

    using "$TABLES_DIR/q7_unbalanced_twfe_transformed_variables.xlsx", ///
    sheet("twfe_unbal") firstrow(variables) replace
* Distribution plots

foreach var in WR LS i {
    qui sum twfe_unbal_`var', detail
    local mu  = r(mean)
    local sig = r(sd)
    twoway ///
        (histogram twfe_unbal_`var', density color(blue%40) bins(30)) ///
        (kdensity  twfe_unbal_`var') ///
        (function  y = normalden(x, `mu', `sig'), range(twfe_unbal_`var')), ///
        title("Unbalanced TWFE distribution of `var'") ///
        xtitle("Unbalanced TWFE: `var'") ytitle("Density") ///
        legend(order(1 "Histogram" 2 "KDE" 3 "Normal"))
    graph export "$FIGURES_DIR/q7_unbalanced_twfe_distribution_`var'.png", replace width(800)
}

di "Question 7 outputs saved."

* ============================================================

* QUESTION 8: COMPARISON OF TRANSFORMED VARIABLES

* ============================================================

* DIFFICULTY: Python builds four separate dataframes and loops over them.

* In Stata we must switch datasets in memory for each transformation.
* We use -preserve/restore- and tempfiles throughout.

use `main_data', clear

* Rebuild between and within for Q8
foreach var in WR LS i {
    by country_id: egen `var'_between = mean(`var')
    gen `var'_within = `var' - `var'_between
}

* --- Q8.2 Summary statistics ---

foreach var in WR LS i {
    foreach trans in between within {
        qui sum `var'_`trans', detail
        di "`trans' `var': N=`r(N)', mean=`r(mean)', sd=`r(sd)'"
    }
}

* FD summary
```

```
use `q7_data', clear
foreach var in WR LS i {
    qui sum d_`var', detail
    di "FD d_`var': N=`r(N)', mean=`r(mean)', sd=`r(sd)'"
}
```

* TWFE summary

use `twfe_data', clear

```
foreach var in WR LS i {
    qui sum twfe_`var', detail
    di "TWFE twfe_`var': N=`r(N)', mean=`r(mean)', sd=`r(sd)'"
}
```

* --- Q8.3 Boxplots ---

* Between: one value per country — single overall boxplot

use `main_data', clear

```
foreach var in WR LS i {
    by country_id: egen `var'_between = mean(`var')
}
```

preserve

```
    by country_id: keep if _n == 1
    foreach var in WR LS i {
        graph box `var'_between, ///
            title("Q8 Between distribution: `var'") yline(0)
        graph export "$FIGURES_DIR/q8_boxplot_between_all_countries_`var'.png", replace width(600)
    }
```

restore

* Within by country

```
foreach var in WR LS i {
    by country_id: egen `var'_within = mean(`var') * 0   // placeholder
    replace `var'_within = `var' - `var'_between
    graph box `var'_within, over(country) ///
        title("Q8 One-way within by country: `var'") yline(0)
    graph export "$FIGURES_DIR/q8_boxplot_within_by_country_`var'.png", replace width(1100)
}
```

* FD by country

```
use `q7_data', clear
foreach var in WR LS i {
    graph box d_`var', over(country) ///
        title("Q8 FD by country: `var'") yline(0)
    graph export "$FIGURES_DIR/q8_boxplot_fd_by_country_`var'.png", replace width(1100)
}
```

* TWFE by country

use `twfe_data', clear

```
foreach var in WR LS i {
    graph box twfe_`var', over(country) ///
        title("Q8 TWFE by country: `var'") yline(0)
    graph export "$FIGURES_DIR/q8_boxplot_twfe_by_country_`var'.png", replace width(1100)
}
```

* --- Q8.4 Correlation matrices ---

* DIFFICULTY: Heatmaps are not native in Stata.

```
* Install -heatplot- from SSC if available: -ssc install heatplot-
```

* Otherwise export to Excel and visualise there.

```
* We export correlation matrices and note the limitation.

* Between correlation matrix (WR, LS, i)

use `main_data', clear

foreach var in WR LS i {
    by country_id: egen `var'_between = mean(`var')
}

preserve

    by country_id: keep if _n == 1
    pwcorr WR_between LS_between i_between, star(.05)
    * DIFFICULTY: -pwcorr- output cannot be directly exported.
    * Export via matrix:
    correlate WR_between LS_between i_between
    matrix C = r(C)
    putexcel set "$TABLES_DIR/q8_between_correlation_matrix.xlsx", replace
    putexcel A1 = matrix(C), names
restore

* Within correlation matrix

use `main_data', clear

foreach var in WR LS i {
    by country_id: egen `var'_between = mean(`var')
    gen `var'_within = `var' - `var'_between
}

correlate WR_within LS_within i_within

matrix C = r(C)

putexcel set "$TABLES_DIR/q8_within_correlation_matrix.xlsx", replace
putexcel A1 = matrix(C), names

* FD correlation matrix

use `q7_data', clear
correlate d_WR d_LS d_i

matrix C = r(C)

putexcel set "$TABLES_DIR/q8_fd_correlation_matrix.xlsx", replace
putexcel A1 = matrix(C), names

* TWFE correlation matrix

use `twfe_data', clear

correlate twfe_WR twfe_LS twfe_i

matrix C = r(C)

putexcel set "$TABLES_DIR/q8_twfe_correlation_matrix.xlsx", replace
putexcel A1 = matrix(C), names

* Heatmap note:

di "NOTE: Heatmap visualisation requires -heatplot- (ssc install heatplot)."

di "      Run: heatplot twfe_WR twfe_LS twfe_i after loading each dataset."

* --- Q8.5 Autocorrelation and trend correlation ---

use `main_data', clear
```

```
xtset country_id year

* Build trend within country

by country_id: gen trend = _n
foreach var in WR LS i {
    * Lag 1
    gen `var'_lag1 = L.`var'
    * Autocorrelation
    qui corr `var' `var'_lag1
    di "Autocorrelation `var': r = `r(rho)'"
    * Trend correlation
    qui corr `var' trend
    di "Trend correlation `var': r = `r(rho)'"
}

* First 30 FD observations with lags

use `q7_data', clear
foreach var in WR LS i {
    gen d_`var'_lag1 = L.d_`var'
}

list country year d_WR d_LS d_i d_WR_lag1 d_LS_lag1 d_i_lag1 in 1/30

export excel country year d_WR d_LS d_i d_WR_lag1 d_LS_lag1 d_i_lag1 ///

    using "$TABLES_DIR/q8_first_30_fd_and_lag_check.xlsx", ///
    sheet("fd_lag") firstrow(variables) replace
* --- Q8.6 Bivariate graphs: linear + quadratic + LOWESS ---

* DIFFICULTY: Python combines three fits in one figure using matplotlib.

* Stata uses -twoway- with (lfit), (qfit), and (lowess) combined.

* Between

use `main_data', clear

foreach var in WR LS i {
    by country_id: egen `var'_between = mean(`var')
}

preserve

    by country_id: keep if _n == 1
    foreach y in WR LS {
        qui corr `y'_between i_between
        local rho = r(rho)
        twoway ///
            (scatter `y'_between i_between, mcolor(black%55)) ///
            (lfit    `y'_between i_between, lcolor(blue))      ///
            (qfit    `y'_between i_between, lcolor(orange))    ///
            (lowess  `y'_between i_between, lcolor(green) bwidth(.3)), ///
            title("Q8 Between: i and `y'" "Corr = `: display %5.3f `rho''") ///
            xtitle("i_between") ytitle("`y'_between") ///
            legend(order(2 "Linear" 3 "Quadratic" 4 "LOWESS"))
        graph export "$FIGURES_DIR/q8_bivariate_between_i_`y'.png", replace width(800)
    }
restore

* Within

foreach var in WR LS i {
    by country_id: egen `var'_within_q8 = mean(`var') * 0
    by country_id: replace `var'_within_q8 = `var' - `var'_between
}

foreach y in WR LS {
    qui corr `y'_within_q8 i_within_q8
    local rho = r(rho)
    twoway ///
```

```
        (scatter `y'_within_q8 i_within_q8, mcolor(black%55)) ///
        (lfit    `y'_within_q8 i_within_q8, lcolor(blue))      ///
        (qfit    `y'_within_q8 i_within_q8, lcolor(orange))    ///
        (lowess  `y'_within_q8 i_within_q8, lcolor(green) bwidth(.3)), ///
        title("Q8 Within: i and `y'" "Corr = `: display %5.3f `rho''") ///
        xtitle("i_within") ytitle("`y'_within") ///
        legend(order(2 "Linear" 3 "Quadratic" 4 "LOWESS"))
    graph export "$FIGURES_DIR/q8_bivariate_within_i_`y'.png", replace width(800)
}

* FD

use `q7_data', clear
foreach y in WR LS {
    qui corr d_`y' d_i
    local rho = r(rho)
    twoway ///
        (scatter d_`y' d_i, mcolor(black%55)) ///
        (lfit    d_`y' d_i, lcolor(blue))      ///
        (qfit    d_`y' d_i, lcolor(orange))    ///
        (lowess  d_`y' d_i, lcolor(green) bwidth(.3)), ///
        title("Q8 FD: d_i and d_`y'" "Corr = `: display %5.3f `rho''") ///
        xtitle("d_i") ytitle("d_`y'") ///
        legend(order(2 "Linear" 3 "Quadratic" 4 "LOWESS"))
    graph export "$FIGURES_DIR/q8_bivariate_fd_i_`y'.png", replace width(800)
}

* TWFE

use `twfe_data', clear

foreach y in WR LS {
    qui corr twfe_`y' twfe_i
    local rho = r(rho)
    twoway ///
        (scatter twfe_`y' twfe_i, mcolor(black%55)) ///
        (lfit    twfe_`y' twfe_i, lcolor(blue))      ///
        (qfit    twfe_`y' twfe_i, lcolor(orange))    ///
        (lowess  twfe_`y' twfe_i, lcolor(green) bwidth(.3)), ///
        title("Q8 TWFE: i and `y'" "Corr = `: display %5.3f `rho''") ///
        xtitle("twfe_i") ytitle("twfe_`y'") ///
        legend(order(2 "Linear" 3 "Quadratic" 4 "LOWESS"))
    graph export "$FIGURES_DIR/q8_bivariate_twfe_i_`y'.png", replace width(800)
}

di "Question 8 outputs saved."

* ============================================================

* QUESTION 9: COUNTRY HETEROGENEITY

* ============================================================

* FD heterogeneity

use `q7_data', clear
foreach y in WR LS {
    tempfile het_fd_`y'
    cap erase `het_fd_`y''
    levelsof country, local(ctry_list)
    foreach ctry of local ctry_list {
        qui count if country == "`ctry'" & !missing(d_`y') & !missing(d_i)
        if r(N) >= 3 {
            qui corr d_`y' d_i if country == "`ctry'"
            local rho = r(rho)
            qui sum d_`y' if country == "`ctry'"
            local sig_y = r(sd)
            qui sum d_i if country == "`ctry'"
            local sig_x = r(sd)
        }
        else {
            local rho   = .
            local sig_y = .
            local sig_x = .
        }
        local slope = cond(!missing(`rho') & `sig_x' != 0, `rho' * `sig_y' / `sig_x', .)
```

```
        clear
        set obs 1
        gen dep_var   = "`y'"
        gen trans     = "First differences"
        gen ctry_name = "`ctry'"
        gen corr      = `rho'
        gen sigma_y   = `sig_y'
        gen sigma_x   = `sig_x'
        gen beta      = `slope'
        gen group     = cond(missing(`rho'), "Missing", ///
                             cond(`rho' > .08, "Positive", ///
                             cond(`rho' < -.08, "Negative", "Weak")))
        cap append using `het_fd_`y''
        save `het_fd_`y'', replace
    }
}
```

* TWFE heterogeneity

use `twfe_data', clear

```
foreach y in WR LS {
    tempfile het_twfe_`y'
    cap erase `het_twfe_`y''
    levelsof country, local(ctry_list)
    foreach ctry of local ctry_list {
        qui count if country == "`ctry'" & !missing(twfe_`y') & !missing(twfe_i)
        if r(N) >= 3 {
            qui corr twfe_`y' twfe_i if country == "`ctry'"
            local rho = r(rho)
            qui sum twfe_`y' if country == "`ctry'"
            local sig_y = r(sd)
            qui sum twfe_i if country == "`ctry'"
            local sig_x = r(sd)
        }
        else {
            local rho   = .
            local sig_y = .
            local sig_x = .
        }
        local slope = cond(!missing(`rho') & `sig_x' != 0, `rho' * `sig_y' / `sig_x', .)
        clear
        set obs 1
        gen dep_var   = "`y'"
        gen trans     = "Two-way fixed effects"
        gen ctry_name = "`ctry'"
        gen corr      = `rho'
        gen sigma_y   = `sig_y'
        gen sigma_x   = `sig_x'
        gen beta      = `slope'
        gen group     = cond(missing(`rho'), "Missing", ///
                             cond(`rho' > .08, "Positive", ///
                             cond(`rho' < -.08, "Negative", "Weak")))
        cap append using `het_twfe_`y''
        save `het_twfe_`y'', replace
    }
}
```

* Combine all heterogeneity results

use `het_fd_WR', clear

append using `het_fd_LS'

append using `het_twfe_WR'

append using `het_twfe_LS'

sort dep_var trans corr

```
export excel using "$TABLES_DIR/q9_country_heterogeneity_fd_twfe.xlsx", ///
    sheet("heterogeneity") firstrow(variables) replace
```

* Group diagnosis

contract dep_var trans group, freq(n_countries)

```
export excel using "$TABLES_DIR/q9_group_diagnosis_summary.xlsx", ///
    sheet("diagnosis") firstrow(variables) replace
di "Question 9 outputs saved."

* ============================================================

* QUESTION 10: PANEL ESTIMATORS

* ============================================================

* DIFFICULTY: Python uses -linearmodels- which mirrors Stata's panel

* commands closely. The Mundlak CRE uses -xtreg, re- with group means

* added manually — identical in logic to the Python version.

use "$CLEAN_DATA_FILE", clear

encode country, gen(country_id)

xtset country_id year

* Drop rows missing any of the key variables

drop if missing(LS) | missing(WR) | missing(i) | missing(GDP) | missing(UN) | missing(REER)
foreach dep_var in LS WR {
    di _newline(2)
    di "====================================================="
    di "RESULTS FOR DEPENDENT VARIABLE: `dep_var'"
    di "====================================================="
    * ------------------------------------------------------------
    * 1. Between estimator
    * ------------------------------------------------------------
    di _newline "======== BETWEEN ESTIMATOR ========"
    xtreg `dep_var' $EXPL_VARS, be
    estimates store between_`dep_var'
    cap log using "$TABLES_DIR/q10_between_results_`dep_var'.txt", text replace
        xtreg `dep_var' $EXPL_VARS, be
    cap log close
    * ------------------------------------------------------------
    * 2. Fixed effects (one-way within)
    * ------------------------------------------------------------
    di _newline "======== FIXED EFFECTS (WITHIN) ========"
    xtreg `dep_var' $EXPL_VARS, fe
    estimates store fe_`dep_var'
    cap log using "$TABLES_DIR/q10_fe_results_`dep_var'.txt", text replace
        xtreg `dep_var' $EXPL_VARS, fe
    cap log close
    * ------------------------------------------------------------
    * 3. Two-way fixed effects
    * DIFFICULTY: Stata's -xtreg, fe- absorbs entity effects only.
    * For TWFE add time dummies explicitly or use -areg- with -absorb-.
    * Here we use -reghdfe- (SSC) which handles both efficiently.
    * Alternative without reghdfe: add i.year dummies manually.
    * ------------------------------------------------------------
    di _newline "======== TWO-WAY FIXED EFFECTS ========"
    cap which reghdfe
    if _rc == 0 {
        reghdfe `dep_var' $EXPL_VARS, absorb(country_id year)
        estimates store twfe_`dep_var'
    }
    else {
        di "NOTE: -reghdfe- not installed. Falling back to xtreg with year dummies."
        di "      Install with: ssc install reghdfe"
        xi: xtreg `dep_var' $EXPL_VARS i.year, fe
        estimates store twfe_`dep_var'
    }
    cap log using "$TABLES_DIR/q10_twfe_results_`dep_var'.txt", text replace
        estimates replay twfe_`dep_var'
    cap log close
    * ------------------------------------------------------------
    * 4. First differences
    * ------------------------------------------------------------
    di _newline "======== FIRST DIFFERENCES ========"
    preserve
        foreach var in `dep_var' $EXPL_VARS {
```

```
            gen d_`var'_q10 = D.`var'
        }
        reg d_`dep_var'_q10 d_i_q10 d_GDP_q10 d_UN_q10 d_REER_q10, nocons
        estimates store fd_`dep_var'
    restore
    cap log using "$TABLES_DIR/q10_fd_results_`dep_var'.txt", text replace
        estimates replay fd_`dep_var'
    cap log close
    * ------------------------------------------------------------
    * 5. Mundlak / Correlated Random Effects
    * ------------------------------------------------------------
    di _newline "======== MUNDLAK RANDOM EFFECTS ========"
    preserve
        foreach var in $EXPL_VARS {
            by country_id: egen `var'_mean = mean(`var')
        }
        xtreg `dep_var' $EXPL_VARS i_mean GDP_mean UN_mean REER_mean, re
        estimates store mundlak_`dep_var'
    restore
    cap log using "$TABLES_DIR/q10_mundlak_results_`dep_var'.txt", text replace
        estimates replay mundlak_`dep_var'
    cap log close
}

di _newline "All estimations completed successfully."

* 1. ONE DATASET IN MEMORY: Stata cannot hold multiple dataframes

*    simultaneously. The script uses tempfiles and preserve/restore

*    throughout, which adds complexity compared to Python.

*

* 2. SCATTER WITH MARGINAL HISTOGRAMS: Not natively available.

*    Produced as plain scatterplots with linear fit. The -plotplain-

*    or -marginsplot- community commands do not replicate this exactly.

*

* 3. CORRELATION HEATMAPS: No native heatmap. Install -heatplot-

*    from SSC (-ssc install heatplot-) for an approximate equivalent.
*

* 4. TWO-WAY FIXED EFFECTS: Requires -reghdfe- (SSC) for clean output.
*    Without it, the script falls back to xi: xtreg with i.year dummies,
*   which works but is slower and produces a large coefficient table.

*

* 5. CONSECUTIVE BLOCKS: Python's consecutive_blocks() is implemented

*    manually using by-group run-length counters, which is more verbose.

*

* 6. DYNAMIC PATH RESOLUTION: Python uses __file__ to find the script

*    location. Stata requires the user to hard-code BASE_DIR at the top.

*

* 7. EXCEL EXPORT: Requires -putexcel- or -export excel-; matrix export

*    for correlation tables uses -putexcel- which needs Stata 13+.
*
```

```
* 8. SKEWNESS / KURTOSIS IN DETAIL: Available via -sum, detail- but
*    not exported as a table automatically; manual extraction needed.
*
* ============================================================
```

Question 7 - First Differences and Two-Way Fixed Effects Prompts and Answers

Question 7 Prompt 1

Prompt: How can I check that first differences are computed within each country?

Answer

Sort the panel by country and year, then compute the difference within each country group.

```
df = df.sort_values(["country", "year"])


for var in ["LS", "WR", "i"]:
    df[f"d_{var}"] = df.groupby("country")[var].diff()
```

A small boundary check can be created by selecting the first observations of each country. The first difference should be missing for the first year of each country.

```
country_start_rows = df.index[
    df["country"].ne(df["country"].shift())
].tolist()


rows_to_check = []


for row in country_start_rows:
    rows_to_check.extend([row, row + 1])


rows_to_check = sorted(set(
    row for row in rows_to_check if 0 <= row < len(df)
))


boundary_check = df.loc[
rows_to_check,
    ["country", "year", "LS", "d_LS", "WR", "d_WR", "i", "d_i"]
]


boundary_check.to_excel(
TABLES_DIR / "q7_first_difference_boundary_check.xlsx",
    index=False
)
```

Question 7 Prompt 2

Prompt: How can I create a reusable function to plot transformed variable distributions?

Answer

A function can be used to plot the histogram, kernel density estimate and normal curve with the same mean and standard deviation.

```
def save_distribution(series, title, filename, xlabel):
    values = (
```

series

```
        .replace([np.inf, -np.inf], np.nan)
        .dropna()
    )


    mean = values.mean()
    std = values.std()
    x_values = np.linspace(values.min(), values.max(), 300)


    plt.figure(figsize=(8, 5))
    plt.hist(values, bins=30, density=True, alpha=0.5, label="Histogram")


    kde = gaussian_kde(values)
    plt.plot(x_values, kde(x_values), label="Kernel density")


    normal_density = norm.pdf(x_values, mean, std)
    plt.plot(x_values, normal_density, label="Normal distribution")


    plt.axvline(mean, linestyle=":", label="Mean")
    plt.title(title)
    plt.xlabel(xlabel)
    plt.ylabel("Density")
    plt.legend()
    plt.tight_layout()
    plt.savefig(FIGURES_DIR / filename, dpi=300)
    plt.close()
```

Question 7 Prompt 3

Prompt: How can I compute the two way fixed effects transformation in pandas?

Answer

The balanced TWFE transformation subtracts the country mean and the year mean, then adds back the global mean.

```
def twfe_transform(data, variable):
    country_mean = data.groupby("country")[variable].transform("mean")
    year_mean = data.groupby("year")[variable].transform("mean")
    global_mean = data[variable].mean()


    return data[variable] - country_mean - year_mean + global_mean
```

Question 7 Prompt 4

Prompt: How can I save the common yearly component removed from the interest rate?

Answer

The common yearly component removed from i can be computed as minus the year mean plus the global mean.

```
year_mean_i = twfe_data.groupby("year")["i"].mean()
global_mean_i = twfe_data["i"].mean()
```

```
time_component_i = pd.DataFrame({
    "year": year_mean_i.index,
    "minus_year_mean_plus_global_mean_i": -year_mean_i.values + global_mean_i
})


time_component_i.to_excel(
```
TABLES_DIR / "q7_twfe_time_component_i.xlsx",
```
    index=False
)
```

Question 7 Prompt 5

Prompt: How can I order country boxplots by variance?

Answer

Compute the variance by country and use the resulting order in the boxplot.

```
country_order = (
```
plot_data
```
    .groupby("country")[variable]
```
.var()
```
    .sort_values(ascending=True)
```
.index
```
)


values_by_country = [
    plot_data.loc[plot_data["country"] == country, variable]
```
for country in country_order
```
]
```

Question 7 Prompt 6

Prompt: How can I compute a country-level correlation table with a simple slope?

Answer

For each country, compute the correlation, the standard deviation of Y, the standard deviation of X, and the simple slope.

```
def country_correlation_table(data, x_col, y_col):
    rows = []


    for country, country_data in data.groupby("country"):
        sample = (
            country_data[[x_col, y_col]]
            .replace([np.inf, -np.inf], np.nan)
            .dropna()
        )


        if len(sample) < 3 or sample[x_col].nunique() <= 1:
```
    continue

```
        x = sample[x_col]
        y = sample[y_col]


        correlation = x.corr(y)
        std_x = x.std()
        std_y = y.std()
        simple_slope = correlation * std_y / std_x


        rows.append({
            "country": country,
            "N": len(sample),
            "correlation": correlation,
            "std_y": std_y,
            "std_x": std_x,
            "simple_slope": simple_slope,
        })


return (
        pd.DataFrame(rows)
        .sort_values("correlation", ascending=False)
        .reset_index(drop=True)
    )
```

Question 7 Prompt 7

Prompt: How can I compute TWFE transformed variables for an unbalanced panel?

Answer

For an unbalanced panel, remove country averages first, then remove year averages from the within transformed variable.

```
for var in ["LS", "WR", "i"]:
    country_mean = unbalanced_data.groupby("country")[var].transform("mean")
    within_var = unbalanced_data[var] - country_mean


    year_mean = within_var.groupby(unbalanced_data["year"]).transform("mean")


    unbalanced_data[f"twfe_unbalanced_{var}"] = within_var - year_mean
```

The transformed variables can then be exported.

```
unbalanced_data[
    ["country", "year", "LS", "WR", "i",
     "twfe_unbalanced_LS",
     "twfe_unbalanced_WR",
     "twfe_unbalanced_i"]
].to_excel(
TABLES_DIR / "q7_unbalanced_twfe_transformed_variables.xlsx",
    index=False
)
```